

Jiang, J., Li, Z., Qin, H., Jiang, M., Luo, X., Wu, X., Wang, H., Tang, Y., Qian, C. and Chen, T. (2024) Unearthing gas-wasting code smells in smart contracts with large language models. *IEEE Transactions on Software Engineering*, (doi: [10.1109/TSE.2024.3491578](https://doi.org/10.1109/TSE.2024.3491578)).

There may be differences between this version and the published version. You are advised to consult the publisher's version if you wish to cite from it.

<http://eprints.gla.ac.uk/339786/>

Deposited on 21 October 2024

Enlighten – Research publications by members of the University of Glasgow
<http://eprints.gla.ac.uk>

Unearthing Gas-Wasting Code Smells in Smart Contracts with Large Language Models

Jinan Jiang, Zihao Li, Haoran Qin, Muhui Jiang, Xiapu Luo, Xiaoming Wu, Haoyu Wang, Yutian Tang, Chenxiong Qian, Ting Chen

Abstract—Smart contracts are automated programs stored on a blockchain, featuring unique attributes such as permissionlessness, trustlessness, immutability, and transparency. These properties underpin an array of unprecedented decentralized services. Compiled into bytecodes, Ethereum smart contracts are executed within the Ethereum Virtual Machine (EVM). Ethereum’s distinct gas mechanism assigns a price to each byte-code execution, incentivizing resource-efficient computing. However, a disconnect exists between conventional coding practices and the less intuitive gas consumption computation mechanism, resulting in inadvertent gas wastage. Gas-wasting code smells at the source code level have been studied in various related works; however, the task of manually identifying such code smells by reading through codes and reasoning about them is both time-consuming and economically inefficient.

In this work, we propose to leverage Large Language Models (LLMs), which have seen a surge in popularity recently, to facilitate undertaking the labor-intensive part of the code-smell-finding pipeline. In particular, we focus on Solidity, the predominant programming language for Ethereum smart contracts. Overall, we identified 26 gas-wasting code smells, out of which 13 were not presented in previous papers. On average, applying these code smells led to a reduction of approximately 10.534% in deployment costs and 21.528% in message call costs across our test codes. We further make a report on each of the identified code smells with associated example contracts sourced from either previous literature or recently deployed contracts.

Index Terms—Artificial Intelligence, Patterns, Program Analysis, Smart Contracts, Language Models.

I. INTRODUCTION

SMART contracts are automated programs running on blockchains, possessing unique properties of being permissionless, trustless, immutable, and transparent. These properties stem from their design, rooted in decentralization and cryptographic security. Consequently, they enable a wide variety of applications across various fields. Some of these applications echo traditional banking systems, such as exchanges and lending, while others introduce novel concepts, such as decentralized finance (DeFi) without central moderators, Non-fungible tokens (NFTs) that use unique digital identifiers to denote digital asset ownership, and Decentralized Autonomous Organizations (DAOs) where community decisions are made through open discussion and voting in a decentralized manner.

Ethereum is one of the most popular blockchain platforms supporting smart contracts. Its smart contracts are usually developed in high-level programming languages, with Solidity being the most popular one that dominates the Ethereum ecosystem in terms of Total Value Locked (TVL) [1]. The high-level programming languages are compiled into byte-codes that will be executed in the Ethereum Virtual Machine

(EVM) after deployment. Ethereum introduces the gas mechanism, which quantifies the computational resources required for transaction execution, priced in wei units, where 1 wei is equivalent to 10^{-18} Ether, for many purposes, such as avoiding resource abuse. However, there is a misalignment between conventional coding practices and the unique gas-pricing system, often due to a lack of understanding or awareness of the gas mechanism among developers. This misalignment leads to numerous instances where programmers write inefficient codes, wasting computational resources, and incurring higher charges. Although previous studies [2]–[15] have attempted to identify code smells that waste gas, these approaches remain largely labor-intensive as they require manually reading a substantial amount of codes or other supplemental resources such as blog posts. An automated approach to replace these manual efforts, in terms of reading codes and reasoning about their logical flaws, would be ideal. However, the automation of code analysis and defect detection has remained an open challenge due to the inherent complexity of programming codes.

In this paper, we propose to leverage Large Language Models (LLMs) to facilitate undertaking the aforementioned labor-intensive part of the code-smell-finding pipeline because of their remarkable human-like performance across various tasks. However, this is a non-trivial task and employing LLMs in this context is not without its challenges. In particular, direct application of LLMs may encounter common issues like hallucination and faithfulness [16]–[20], requiring a meticulously crafted approach to avoid these pitfalls. Moreover, the construction of the input prompt necessitates a high degree of engineering and careful design. To address these issues, we first propose a new framework for identifying gas-wasting code smells (§III-A) by integrating GPT-4 into the pipeline of code smell identification with the injection of design choices grounded on existing works [17]–[22], [22]–[26]. We then assess the novelty of our identified code smells by comparing our set of 26 identified code smells with existing works to show that 13 of them are new (§IV-B), as well as the effectiveness of our code smells by showing that they could save 10.534 % in deployment costs and 21.528 % in message call costs across our test codes (§IV-C). Further, to gain deeper insights into our code smell identification process, we extend our investigation to show an ablation study (§IV-D), to present the primary challenges we encountered when using GPT-4 (§IV-E), to characterize the importance of the identified code smells (§IV-F), and to examine the extensibility of our proposed pipeline (§IV-G). Lastly, we present several case

study to demonstrate the identified code smells (§V).

Overall, our contributions are summarized as follows:

- 1. A new set of identified gas-wasting code smells at the source code level:** To our best knowledge, our identified set of 26 code smells is the largest among existing work [2], [4]–[6], [8], [9], [11], [12], where 13 of our identified code smells have not been presented in previous works. This list could be referenced by both smart contract developers and researchers.
- 2. A novel approach of leveraging GPT-4 into the pipeline of code smell identification:** To our best knowledge, existing works have been largely relying on manual efforts to identify gas-wasting code smells. We are the first to integrate an LLM to automate this task, which we show to be highly effective in terms of reducing manual efforts. In addition, by grounding on existing research works, we have carefully considered GPT-4’s strengths (e.g., long-range reading and summarizing) and weaknesses (e.g., faithfulness and reliability issues). This provides a roadmap for future works in terms of the integration of LLMs like GPT-4 into complex systems.
- 3. Presentation of comprehensive analysis and insights:** To unearth further insights about our identified code smells and the process of identifying them, we perform further studies including an ablation study, quality assessments, characterization of the importance of code smells, an examination of the extensibility of our proposed pipeline, discussion of the implications of toy contracts, and so on. We believe these insights will assist Solidity developers in better understanding gas optimization techniques and guide future research on utilizing LLMs to facilitate task completion.

II. BACKGROUND

A. Execution of a transaction and gas fees.

In this section, we introduce the concept of transaction execution as well as the gas fees mechanism of Ethereum [27]. The Ethereum Virtual Machine (EVM) is a transaction-based state machine, where each transaction constitutes a state transition, formulated as $\delta' = \Upsilon(\delta, T)$, where Υ is the state transition function, δ is the state, T is a transaction, and δ' is the post-transactional state. In particular, transactions could trigger the execution of bytecode instructions on the EVM, and each instruction consumes a certain amount of gas, determined by a pre-defined fee schedule. A selected subset of the fee schedule [27], [28], which are the ones that are more relevant to the code smells discussed in this paper, is shown at Table I. The structure of these fees is designed to reflect the computational and storage demands placed on the network. For example, operations involving blockchain storage (e.g., `SSTORE`) are given a higher price due to their permanent impact on the blockchain state and greater resource consumption compared to the ephemeral memory operations. This pricing strategy helps manage the network’s resources efficiently, ensuring that more resource-intensive operations are associated with higher costs. It is worth noting that certain operations are also allowed to receive a gas refund, such as when non-zero storage values are reset to zero, encouraging the optimization of storage usage by clearing unused or unnecessary storage space. Overall, with such a complex gas

TABLE I: An example gas fee schedule.

Name	Value	Descriptions
G_{zero}	0	Paid for <code>STOP</code> , <code>RETURN</code> and <code>REVERT</code> .
G_{verylow}	3	Paid for <code>ADD</code> , <code>SUB</code> , <code>NOT</code> , <code>LT</code> , <code>GT</code> , <code>EQ</code> , <code>ISZERO</code> , <code>AND</code> , <code>OR</code> , <code>XOR</code> , <code>SHL</code> , <code>SHR</code> , <code>CALLDATACOPY</code> , <code>MLOAD</code> , <code>MSTORE</code> , <code>MSTORE8</code> , <code>PUSH*</code> , <code>DUP*</code> , <code>SWAP*</code> , etc.
G_{low}	5	Paid for <code>MUL</code> , <code>DIV</code> , <code>SDIV</code> , <code>MOD</code> , etc.
G_{mid}	8	Paid for <code>ADDMOD</code> , <code>MULMOD</code> and <code>JUMP</code> .
G_{high}	10	Paid for <code>JUMPI</code> .
$G_{\text{warmaccess}}$	100	Cost of a warm account or storage access.
G_{coldload}	2100	Cost of a cold storage access.
G_{sreset}	2900	Paid for an <code>SSTORE</code> operation when the storage value’s zeroness remains unchanged or is set to zero.
G_{sset}	20000	Paid for an <code>SSTORE</code> operation when the storage value is set to non-zero from zero.
R_{sclear}	-4800	Refund given (added into refund counter) when the storage value is set to zero from non-zero.
G_{logdata}	8	Paid for each byte in a <code>LOG</code> operation’s data.
G_{log}	375	Partial payment for a <code>LOG</code> operation.
G_{logtopic}	375	Paid for each topic of a <code>LOG</code> operation.
$G_{\text{txdatazero}}$	4	Paid for every zero byte of data or code for a transaction.
$G_{\text{txdatanonzero}}$	16	Paid for every non-zero byte of data or code for a transaction.
$G_{\text{transaction}}$	21000	Paid for every transaction.
$G_{\text{codedeposit}}$	200	Paid per byte for a <code>CREATE</code> operation to succeed in placing code into state.
G_{create}	32000	Paid for a <code>CREATE</code> operation.

pricing mechanism, there comes with various opportunities of identifying inefficiencies in gas usage, which are called gas-wasting code smells or gas-wasting patterns [2], [4]–[6], [8], [9], [11], [12], [29], [30], which is the primary focus of this study.

B. LLM: strength and weakness of GPT-4

Ever since the introduction of the transformer architecture [31], the field of language model has gradually taken the transition from traditional models (e.g. n-grams and recurrent neural networks) to the nowadays increasingly larger and more complex architectures. Some of the more recent models from the year of 2023 include GPT-4 [16], CodeGen [32], WizardCoder [33], StarCoder [32], LLaMA [34], etc. Among them, GPT-4 has gained tremendous popularity due to its strength in its human-like performance in conducting tasks ranging from everyday conversations to even complex logical reasonings, and it is the focus of our work. However, GPT-4 comes with limitations as well, and we need to be aware of both its strengths and weaknesses of it in order to better harness its power. In particular, according to its developers [16], GPT-4 has the advantage of:

- 1) Performing human-like conversations and basic reasonings.
- 2) Predictable behavior (e.g. loss, performance on testing datasets, etc) across multiple scales.
- 3) Human-level performance on a wide range of professional and academic exams.
- 4) Significantly outperforming existing language models.

5) Outstanding capability to follow user intent.

On the other hand, some of its limitations include:

- 1) Fact hallucination and reasoning errors: GPT-4 could produce contents that is nonsensical or untruthful. This contributes to a risk of reliability, and great care is needed when processing outputs from GPT-4.
- 2) Could be overly gullible in accepting obviously false statements from a user, and could be confidently wrong in its predictions.

Note that these limitations imply that we should exert caution when processing outputs from GPT-4, which may not be fully reliable.

III. METHODOLOGY

In this section, we describe the overall methodology that we took to approach the task of LLM-facilitated identification of gas-wasting code smells. To begin with, we define some terminology that we will use.

Reported code smells: by reported code smells, we mean the code smells that are reported by GPT-4¹, which may or may not be correct.

Identified code smells: by identified code smells, we mean the code smells that have passed our verification and are deemed valid. We use it interchangeably with "found code smells."

In the literature, the terms *pattern* and *code smells* are used variably to describe problematic coding issues. We opt for *code smells* to differentiate from issues that are already well-established and universally recognized, which are typically implied by the term *pattern*. This is because our research aims for unearthing novel and context-specific issues that were not previously reported, making the term *code smells* better align with our study.

Overall, we first introduce the pipeline we adopted to identify the new code smells (§III-A). Then we introduce further details about the prompt framework (§III-B), categorization procedure of code smells (§III-C), as well as the evaluation metrics (§III-D).

A. The Code Smell Identification Pipeline

In this section, we introduce the design of our pipeline for the process of identifying gas-inefficient code smells.

It's important to highlight that traditionally, the identification of such code smells has been a challenging task [2], [4]–[6], [8], [9], [11], [12], necessitating domain experts to painstakingly read and reason about large volumes of code, especially when dealing with extensive codes with complex dependencies. This is not only time-consuming but also economically inefficient. On the other hand, the advent of GPT-4 provides an unprecedented opportunity to emulate extensive manpower that could be used to replace the aforementioned labor-intensive tasks via API calls, which was impossible before. In particular, some of the advantages of GPT-4 include its human-level performance on a wide range of tasks and its capability to follow user intent.

However, GPT-4 comes with its own issues. Specifically, we should not fully trust the outputs of LLMs and we have to exert enough caution as not to be trapped in the faithfulness or other reliability issues of GPT-4 [16], [17], [19], [20]. As discussed in Section II-B, GPT-4 has the issue of fact hallucination and reasoning errors where it could produce contents that is nonsensical or untruthful, the issue of being prone to be overly gullible in accepting obviously false statements from a user, as well as the issue of potentially being confidently wrong in its predictions. With these defects in mind, we instead delegate to GPT-4 only the labor-intensive portions of the task, such as code reading, logical reasoning, and preliminary code smell reporting. Overall, this has the benefit of addressing the limitations of GPT-4 as discussed at Section II-B.

The overall workflow is depicted in Figure 1. Note that as shown in the figure, M stands for the number of contracts that are processed in each round, and N represents the number of rounds that are taken. At a high level, we provide input codes to GPT-4 via carefully constructed prompts, GPT-4 then reads through the codes and reports a set of code smells, we verify the reported code smells, and compile the code smells together into categories. In addition, we also keep track of the reasons of how GPT-4 failed at reporting correct code smells.

Below we discuss the major steps of our pipeline.

STEP 1. Code Dataset Crawling and Preprocessing: As a first step, we crawled the recently verified contract codes from Etherscan [36], aiming to obtain a preliminary set of contract codes to sample from for the subsequent steps of the pipeline. In total, we obtained 11,348 Solidity smart contracts from August 2023. Crawling the more recently verified contracts has the benefit of being associated with a higher chance of focusing our work on the more up-to-date Solidity compiler version as well as coding practices. It is worth noting that in practice, developers could specify the version of the Solidity compiler via a pragma directive at the top of the code file, meaning that our selected contracts do not necessarily guarantee that all obtained contracts are written in the newest versions of Solidity. To better understand this, we collected the compiler version from both our collected Solidity and Vyper contracts (Vyper contracts are discussed in more detail at Section IV-G1). Overall, we find that 95 % of the Solidity contracts are written in the newest compiler version (i.e., Solidity 0.8.* versions), and this number is 99.6 % for the Vyper contracts (i.e., Vyper 0.3.* versions). This signifies that our crawled codes are mostly comprised of contracts from more recent compiler versions. Since there are duplicate deployments of the same contract, we performed hash-based deduplication. This is done by computing the hashcode of each of the contract code files, and removing those that are duplicates. As a result, this filtering reduces the number of contracts from 11,348 to 6,614. This part is labeled as "STEP 1" in Figure 1.

STEP 2. Preliminary Code Smell Identification and Categorization: At this step, we aim to verify and report an initial set of gas-wasting code smells, as well as to experiment with different forms of input code blocks. In particular, we applied our initialized prompt framework on a small scale ($M = 10$, $N = 3$, which means 30 contracts in total), where

¹The version we used is gpt-4-0613 [35], which is a snapshot of gpt-4 from June 13th 2023

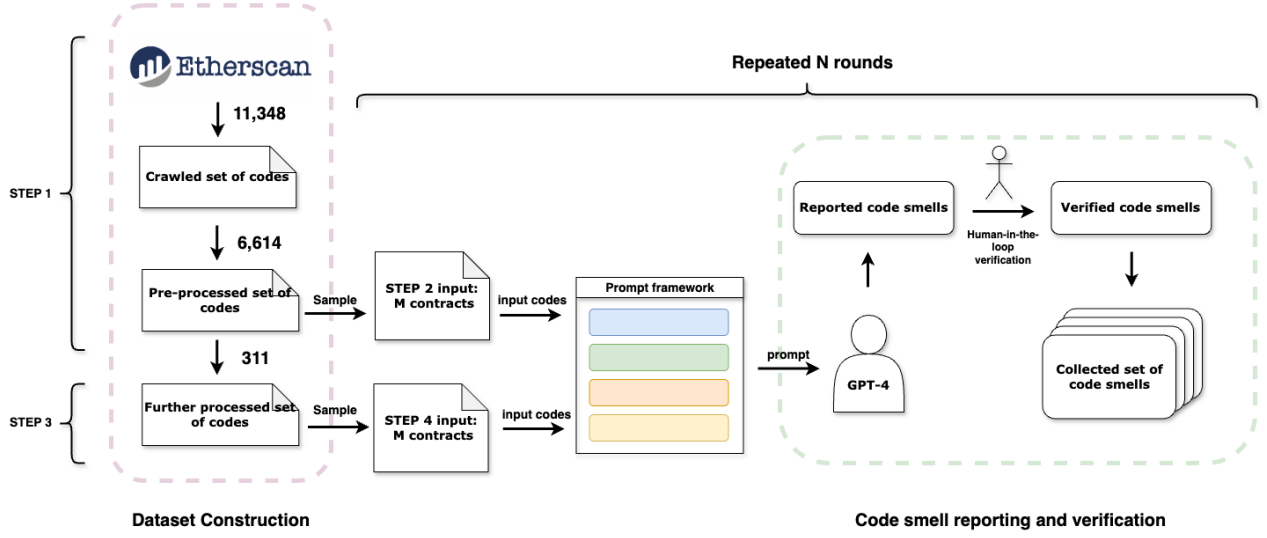


Fig. 1: Illustration of the pipeline.

TABLE II: Statistics of each round of GPT-4-facilitated gas-finding.

Round_num	$\mathcal{P}_{pass}$	$\mathcal{P}_{rej}$	$\mathcal{N}_{pass}$	$\mathcal{N}_{rep}$	$\mathcal{N}_{c_pass}$	$\mathcal{N}_{c_rej}$	$\mathcal{E}_{pass}$	$\mathcal{E}_{rej}$	$\mathcal{A}_{in_ins}$	$\mathcal{P}_{val}$
1	0.455	0.545	10	22	10	4	3.322	1.551	162.6	0.8
2	0.103	0.897	3	29	3	5	1.585	1.676	136.8	0.2
3	0.273	0.727	6	22	3	5	1.459	1.921	62.0	0.3
4	0.423	0.577	11	26	6	6	2.3	2.226	169.9	0.7
5	0.385	0.615	10	26	3	5	1.295	1.936	152.7	0.5
6	0.375	0.625	12	32	5	7	1.781	2.48	191.2	0.8
7	0.385	0.615	10	26	4	6	1.357	2.274	168.7	0.8
8	0.292	0.708	7	24	4	7	1.664	2.343	179.0	0.6

the 10 contracts at each round are sampled from our contract dataset (labeled as "STEP 2 input" in Figure 1). The assembled prompt framework is directly input into GPT-4. Specifically, for the first round, because the entire contract files are too long to be fed as input into GPT-4, we manually remove simple and straightforward components (e.g. short functions, comments, variable declarations) from each contract, in order to preserve the more complex part of the codes that are more likely to contain gas-related issues. However, this is not sustainable as it requires too much manual efforts. Therefore, we need to eliminate the manual efforts of the code selection from round 1. An existing model (i.e. CodeGen) [32] performed code selection by extracting fixed-length continuous chunks from the code dataset. In our second round, we similarly follow this approach by extracting the topmost range from the codes, in the form of taking the non-trivial functions sorted by their order of appearance starting from the first function of each contract. By non-trivial, we mean functions that contain at least four lines of code. Further at the third round, we also experimented with taking the continuous chunk at the function level (i.e. taking the entire codes of a function), and use one entire function as the input codes. In particular, we only sampled functions with more than 1000 characters for higher complexity in the input codes. After getting outputs

from GPT-4, we perform verification by checking the reported code smells along with the corresponding codes to examine if the code smell is valid. A more detailed descriptions of the categorization of code smells is discussed at Section III-C. The statistics of this process for the first 3 rounds, deploying the evaluation metrics as discussed in Section III-D, are shown in the first 3 rows of Table II. From this table, we could observe from Table II that the first round has a noticeably higher quality in terms of the pass rate ($\mathcal{P}_{pass}$), number of verified categories ($\mathcal{N}_{c_pass}$), as well as the percentage of the contracts from which valid code smells are found ($\mathcal{P}_{val}$). These metrics are discussed in more detail at Section III-D. This could be explained by the higher accuracy and quality of the selected codes in a manual approach compared to a naively automated one. Overall, a set of 11 initial code smell categories is identified in this step.

STEP 3. Dataset Processing: Next, we discuss some of the issues we encountered during STEP 2 and the corresponding three further filterings we made to the dataset (this part is labeled as "STEP 3" in Figure 1):

- 1) *Minimum line-of-code requirement.* First, some of the crawled contracts are too short to be a strong candidate. As demonstrated by prior work [37], a higher number of lines-of-codes (LOC) is associated with more defect

proneness (in our case, gas-wasting code smells). To this end, we filtered out contracts with less than 100 lines of codes (LOC) in order to focus our resources on the most promising codes. This reduces the number of contracts from 6,614 to 4,777.

- 2) *Contract-level redundancy*. Second, we observed redundancy at the level of contract names (e.g. different versions of the same contract). For example, out of the 4,777 remaining contracts, there are 251 different versions of the contract "ERC20", where they may exhibit similar behaviors and result in redundancies at the level of functionality. Since we want to put our focus on a set of contracts that are as diverse as possible to exhibit more diverse gas-wasting code smells, we only keep one version of each contract name (by sampling). This filtering brings the number of contracts from 4,777 down to 3,372.
- 3) *Function-level redundancy*. Third, we observe redundancies at the function level. To be more specific, we found that quite a few function names are repetitively implemented across different contracts. For example, 3,047 implementations of the `_transfer` function is found among the remaining contracts, and this number is 2,605 for `transferFrom` as well as 2,583 for `approve`. To reduce this redundancy, we only kept contracts that contain *salient functions*. By *salient functions*, we mean the functions whose name is unique among all functions and are the top 10 percent of all functions in terms of code length. Recall that code length is positively associated with complexity [37]. We found 709 such *salient functions*, and the filtering at this step reduces the number of contracts from 3,372 to 311.

At a high level, the rationale behind these filters is to prioritize diversity as well as complexity in the functions and contracts that are being analyzed, which is intended to enhance the chance of discovering more varied and less repetitive gas-wasting code smells.

STEP 4. Further Code Smell Discovery: At step 2, the feedback we obtained via the self-reflection block manifests that GPT-4 struggles with a lack of focus in round 2 due to the existence of too many unrelated components (e.g. header comments, imports, etc) and a lack of contextual information in round 3. In order to reduce unrelated components and preserve context, we change the construction of the input code block from top-to-bottom greedy sampling over a continuous range to instead first split a contract into single functions, only keep functions that contain at least 5 lines of codes, rank functions by length, and then greedily sample longest functions into the input code block until reaching the input length limit. These selected functions are synthesized together to make up the input contract for the code input block. This refactoring approach is inspired by the "move method" of standard refactoring techniques [38]. This refinement offers several benefits. First, prioritizing functions with higher LOC has the benefit of a higher chance of encountering complex codes [37], which in turn raises the probability of identifying gas inefficiencies. Second, prioritizing longer functions ensures a

wider in-function context, which addresses the corresponding feedback regarding the lack of context. Third, this automates the manual efforts in the round 1 trial, where simple functions were removed by hand from each contract, which was deemed an unsustainable approach due to a high demand on manual efforts. We apply the updated method of input code block construction to discover further gas-wasting code smells. In particular, we set $N=5$ and $M=10$ (i.e. 50 contracts are sampled from the dataset in total), accounting for rounds 4 through 8, and this part is labeled as "STEP 4 input" in Figure 1). It can be observed from Table II that our updated input code block at this step (i.e. rounds 4 through 8) has better performance than rounds 2 and 3, and is closer to round 1 across various metrics. This shows that our refactoring approach to input code block construction achieves better performance. In total, 18 code smells categories are reported up to round 8.

In summary, building on the insights and lessons distilled from trials as well as model feedback, this step further automates the process to enhance the efficiency of code smell discovery. It diversifies the selection of functions and contracts, aiming to maximize the inclusion of a more varied and less repetitive set of codes. The integration of the knowledge distillation [39] and the feedback mechanism collectively facilitates our prompt tuning process. In particular, the feedback mechanism has been shown to be effective in many existing studies, including reinforcement learning [40], error correction in LLM outputs [41], LLM fine-tuning [42], text summarization [43], and so on. Moreover, additional studies [44]–[46] highlight the importance of feedback mechanisms in maintaining high-quality outputs in code-related tasks.

STEP 5. Ablation: To further explore how our design choices contributed to the identified code smells, we perform an additional round of ablation study. This has the benefit of providing explainability to our methodology. In particular, we took the last 20 contracts from the last two rounds (i.e. round 7 and 8), and applied ablation on them by removing one design component at a time. Further details and results of the ablation study are discussed in the third research question (RQ3, Section IV-D). At the same time, we also keep track of newly found gas-wasting code smells, and add them to our set of code smells. Up to this point, a total of 26 gas-wasting code smells are identified.

STEP 6. Quality Assessment: Overall, we have obtained 26 gas-wasting code smell categories, and we wish to further assess their quality. In particular, this is done in terms of comparing our code smells to previous works to assess novelty and deploying and running optimized contracts to record the amount of saved gas. Further details and findings are presented in the first and second research questions (Section IV-B and IV-C).

Summary. Overall, this six-step pipeline allows us to leverage GPT-4 to systematically verify and identify a set of 26 gas-inefficient code smells. In addition, we perform an ablation study as well as quality assessments to demonstrate the effectiveness of our design choices as well as identified code smells. Regarding the human verification performed at the human-in-the-loop step, two of the authors were involved. The

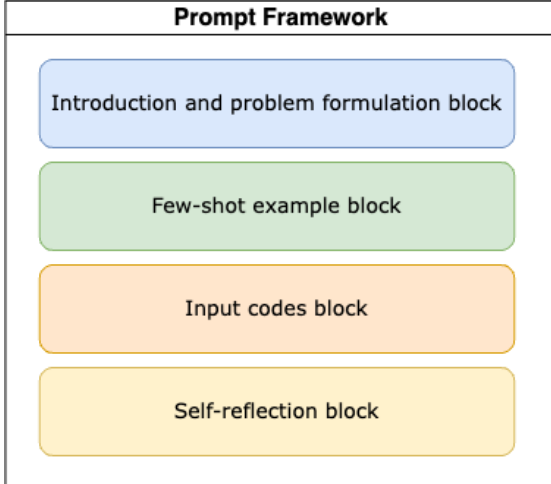


Fig. 2: The prompt framework.

authors possess four and two years of experience in blockchain development, respectively. The first author began by performing the entire verification procedure, and compiled a list of categorization following the steps outlined at Section III-C. The other author then independently repeated the same entire procedure following the same steps, building on the produced categorization C (i.e., as introduced at Algorithm 1) of the first author, without any knowledge of the first author's labeled category of reported code smells. The results from the two authors are then compared, and there is an overall 19.32 % disagreement. Results from the two authors that agree with each other were kept. For each of the conflicts between the two authors, they first independently review and check their initial reasoning process, without knowing the other author's labeled category, and make corrections if mistakes are identified. Then, if the conflict still persists, the two authors would discuss together and check the detailed code smell descriptions and codes to make the final decision to resolve discrepancies, ensuring a robust verification process.

B. Prompt Framework

At this section, we introduce our concept of a *prompt framework*. A *prompt framework* is a composite structure consisting of several blocks, with each one embodying a unique rationale in its design. Its configuration is shown in Figure 2, and we will discuss the blocks in more detail below. Overall, when used as input to GPT-4, all of the blocks are combined into one integrated input prompt.

To begin with, we discuss the major design motivations behind our utilization of GPT-4. By grounding on existing research papers [17]–[22], [22]–[26], we get to leverage various performance-improving techniques from existing works into our code-smell-finding pipeline.

Motivation 1: Few shot examples (FSE). The few-shot example technique refers to the utilization of language models

by providing examples of the target task to guide its process of task completion. Without the need of fine-tuning on pre-trained models, the few-shot example technique is resource-efficient in terms of computational resources. As one of the earlier works, [21] explored the technique of few-shot learning on language models, beyond the traditional paradigm of pretraining-finetuning. In particular, it showed that language models with large sizes like GPT-3 could achieve strong performance across many NLP tasks in few-shot settings. Some later works further demonstrated the strong effectiveness of few-shot learning across a wide range of applications (e.g. clinical domain [47], English writing [48], video-to-text [49], etc).

Motivation 2: Chain of Thought (CoT) prompting. As one of the early works, [22] showed that by including a chain-of-thought prompting sequence of intermediate reasoning steps in the input prompt, the performance of LLMs in complex reasoning tasks could be largely improved. Intuitively, by showing an example of how to decompose a similar task step-by-step in a logical manner, we get to elicit the LLM to perform reasoning steps in a similar manner. Some later experiments further confirm the effectiveness of the chain-of-thought prompting technique [23], [24], [26]. However, it is worth noting that there are faithfulness issues associated with CoT, where the provided explanation from the LLM of how it arrived at the answer might not necessarily reflect the true reasoning process [17]. In particular, existing work [18] showed that CoT explanations could systematically misrepresent the true reason for a model's prediction, compromising the faithfulness of the provided chain-of-thought explanations. This suggests that we should be careful with the CoT outputs and avoid over-reliance issues.

Motivation 3: Verbose elaboration. Another challenge to the usage of LLMs is whether the LLM has correctly understood the intent of the input prompt. Existing research show that when under-specified questions or ambiguous user inputs are provided, language models rarely ask for clarifications, and rather output answers with poor performance [19]. In other words, ambiguity in the input texts could drastically affect the model performance in terms of outputting satisfying answers [20]. This implies that key concepts and instructions in the input prompt must be carefully framed to avoid ambiguity, and one of the ways is to explain the key concept twice from different perspectives, which largely reduces the chance of being misunderstood by the LLM and the consequent lower quality of output.

Motivation 4: Self-inspection. Another perspective to understand CoT is that it forces the LLM to reflect on its own inherent reasoning process, and then explain them in a disciplined way, instead of directly outputting the answer. As presented by [22], the CoT technique is capable of eliciting reasoning from LLMs. Another work [25] demonstrated the effectiveness of asking LLMs to perform self-reflection. In our formulation of the concept of self-inspection, we attempt to add instructions to our input prompts that aims at eliciting reasoning and reflection from GPT-4 from multiple perspectives. As a stronger form of CoT, this has the benefit of asking the LLM to holistically inspect its own reasoning and generation process.

Next, we introduce different blocks below:

1. Introduction and problem formulation block: This block presents an introduction of related concepts and formulates the task to be undertaken by injecting our domain knowledge of gas optimization. This sets the context and orients the focus of the code smell identification process. The design of this block employs the *Motivation 2* and *Motivation 3*. In particular, instead of asking for a direct prediction (i.e. outputting a direct answer without explanations), we use the chain-of-thought technique to ask for a step-by-step explanation to elicit more detailed and structured output from GPT-4. In addition, being aware of the limitations discussed in *Motivation 3*, we have intentionally provided some (not so long) verbose elaborations for some of the key concepts (e.g. definition of the task we are asking the LLM to complete and some important requirements) to the GPT-4. In particular, we repeated each explanation twice in different spots of the input prompt (e.g. one at the beginning as an introduction, and one at the ending as a reminder), where each repetition provides an explanation from a different perspective. This way, we reduce the chance of GPT-4 misunderstanding important details in the task formulation.

2. Few-shot example block: For this block, we populate it with a selected set of gas-wasting code smells with associated explanations from either existing papers or our newly found code smells. The design of this block is based on the *Motivation 1*.

3. Input codes block: This block incorporates code-level inputs into the gas-finding prompt, and asks GPT-4 to report gas-wasting code smells from it. Initially we set the input codes block to be the contract codes greedily sampled at the function level, from which we wish to find gas-wasting code smells. Note that the removal of this block would turn the input prompt into a probing of the memory of GPT-4 (Section IV-D).

4. Self-inspection block: This block is designed to encourage the model to reflect on its own task completion process. In particular, the instructions include three perspectives. First, we ask GPT-4 to provide a set of tradeoff scores for each reported code smell in terms of how much the proposed code smell compromises readability, maintainability, and security of the codes. Second, we ask the LLM about how it could do better the next time if provided with the same task in the future. Third, we ask GPT-4 to tell us if there are ambiguities or any other issues in our provided prompt that caused troubles to its task completion. The design of this utilizes the *Motivation 4*.

C. Categorization of code smells

Algorithm 1 outlines the pseudo-algorithm for the procedure used to categorize new code smells iteratively and incrementally. We adopt a methodology similar to the concept of Iterative and Incremental Development (IID) [50]. Specifically, we start from an initial state of an empty categorization for both verified (i.e., C_p) and rejected (i.e., C_f) code smells, and incrementally build it up in the subsequent loop over all reported code smells (line 3). Note that this accounts for both the passed code smells and the reasons of failure for rejected code smells (discussed in more detail at Section IV-E).

Each iteration involves getting a reported code smell (line 3), deciding its relevance (line 4), and integrating it either by fitting it into an existing category if a match is found (line 12), or otherwise by forming a new one (line 17). This method allows for flexibility in absorbing new code smells, with the advantage of adaptability and continuous improvement.

Algorithm 1 Code Smell Categorization Procedure

```

1:  $C_p \leftarrow \emptyset$ 
2:  $C_f \leftarrow \emptyset$ 
3: for  $p \in \text{getReportedCodeSmells}()$  do
4:    $\text{passed} \leftarrow \text{verifyCodeSmell}(p)$ 
5:   if  $\text{passed}$  then
6:      $C \leftarrow C_p$ 
7:   else
8:      $C \leftarrow C_f$ 
9:   end if
10:   $\text{matched} \leftarrow \text{false}$ 
11:  for each category  $c \in C$  do
12:    if  $p.\text{fitsToCategory}(c)$  then
13:       $c \leftarrow c \cup \{p\}$ 
14:       $\text{matched} \leftarrow \text{true}$ 
15:    end if
16:  end for
17:  if not  $\text{matched}$  then
18:     $n \leftarrow \{p\}$ 
19:     $C \leftarrow C \cup \{n\}$ 
20:  end if
21: end for
```

D. Evaluation Metrics

Below we describe the metrics that we deploy to present statistics of each round of our code-smell-finding process.

- 1) **Entropy.** We categorize the found code smells as well as reasons of rejection of each reported code smells from GPT-4. Following this, we use the entropy metric to report the diversity of the categories in each round (e.g. a lower diversity indicate that the set of found code smells is more biased towards certain categories, and a higher one signifies a more even distribution). A naive entropy $H_n(P)$ is defined in Equation 6. However, as a previous work [51] pointed out, the "Normalized Static Entropy" scale is more fair in terms of bringing the entropy values into the same scale. This is done by dividing by the maximum possible entropy (i.e. $\log_2 n$). The definition is shown below, where $c \in \{\text{pass}, \text{rej}\}$, and $\mathcal{E}_{\text{pass}}$ denotes the normalized entropy of the verified code smell categories and $\mathcal{E}_{\text{rej}}$ stands for that of the reasons of rejection categories.

$$H_n(P) = - \sum_{i=1}^n (P_i \times \log_2 P_i) \quad (1)$$

$$\mathcal{E}_c = \frac{1}{\text{Max Entropy for Distribution}} * H_n(P) \quad (2)$$

$$= \frac{1}{\log_2 n} * H_n(P) \quad (3)$$

$$= -\frac{1}{\log_2 n} * \sum_{k=1}^n (p_k \times \log_2 p_k) \quad (4)$$

$$= -\sum_{k=1}^n (p_k \times \log_2 p_k) \quad (5)$$

- 2) **Pass rate.** It means the percentage of the reported code smells that passed the human-in-the-loop verification, and we define it as $\mathcal{P}_{pass}$. We similarly define $\mathcal{P}_{rej}$ as the percentage of rejected code smells. A higher $\mathcal{P}_{rej}$ means that a higher proportion of the reported code smells are deemed invalid, and thus the set of reported code smells overall have a lower quality.
- 3) **Number of code smells.** The number of reported code smells from GPT-4 during a certain round, denoted by $\mathcal{N}_{rep}$. Note that this number could contain duplicates from the same categories. A higher $\mathcal{N}_{rep}$ means more code smells are reported.
- 4) **Number of categories.** The number of verified code smell categories from each round is denoted by $\mathcal{N}_{c_{pass}}$. We similarly denote $\mathcal{N}_{c_{rej}}$ to be the number of reasons of rejection categories. A higher $\mathcal{N}_{c_{pass}}$ means more categories of code smells are identified.
- 5) **Average number of lines.** Average number of lines in the input prompt, denoted by $\mathcal{A}_{in_lns}$. This metric is used to keep track of the size of input prompts.
- 6) **Proportion of valid contracts.** The fraction of contracts from which valid gas code smells were reported, denoted by $\mathcal{P}_{val}$. A higher $\mathcal{P}_{val}$ means that more of the input contracts are identified to have gas-wasting code smells.

IV. EVALUATION

A. Research Questions

In the evaluation section, our goal is to address six research questions that guide our comprehensive analysis. To begin with, given that there are already several existing works that attempt to present gas-wasting code smells, we wish to investigate to what extent our identified code smells overlap with existing ones and how many of our code smells are new. Following this, we encapsulate the first research question as follows:

RQ1: How novel is our identified set of code smells?

We also seek to comprehend the effectiveness of our identified set of code smells, in terms of the amount of gas that could be saved. This leads to our second research question:

RQ2: How much gas does our identified set of code smells save?

In addition, we strive to understand the impact of some of our design choices on the code smell discovery pipeline. A natural approach to this is an ablation study, which we apply in answering the third research question:

RQ3: What impact do our design choices have on our code-smell-finding process and results?

Furthermore, as an endeavor to better understand the code-smell-identifying process, we wish to understand how GPT-4 failed at reporting gas-wasting code smells. This leads to the fourth research question:

RQ4: How did GPT-4 fail when performing the task of identifying gas-wasting code smells?

Additionally, we wish to examine the importance of the discovered code smells from both intrinsic and extrinsic perspectives. This leads to our fifth research question:

RQ5: What is the importance of various features of our identified code smells in smart contracts?

Finally, it is important to also identify the flexibility and adaptability of our framework to application contexts beyond its initial setup. This leads to the sixth research question:

RQ6: What is the extensibility and adaptability of our framework? For example, to different programming languages, to novel EVM features, and to different LLMs?

B. RQ1: Novelty: compared to existing works.

In this section, we aim at evaluating the novelty of the 26 code smells identified during our gas-wasting code-smell-finding procedure. The full list of code smell names is listed in the left-most columns of Table III.

In order to identify a list of existing works related to gas-wasting code smells, we adopt a paper searching methodology similar to that of Systematic Literature Review (i.e., SLR) [52]–[54], which is commonly used in survey papers to identify comprehensive paper lists relevant to a certain topic. This approach has the benefit of ensuring both comprehensiveness and reproducibility in identifying relevant literature. The major steps of our search strategy are outlined below:

- 1) **STEP 1: Search string determination.** To identify papers that are relevant to gas-wasting patterns, we determine the search strings to consist of the combination of the keywords: *Solidity*, *pattern*, *gas*, *wasting*, and *inefficient*. The specific strings we used are:
 - a) *"solidity gas wasting pattern"*
 - b) *"solidity gas wasting"*
 - c) *"solidity gas pattern"*
 - d) *"solidity gas inefficient"*
- 2) **STEP 2: Conducting the search.** Using the above search strings, we conduct the searching on Google Scholar, using its default settings with no filters applied, and relevant papers are identified from the search results.

TABLE III: Comparison of our identified code smells to existing papers. ● means the corresponding paper presented a code smell that is exactly the same as ours, and ○ means the corresponding paper presented a code smell that is tangentially related to ours.

Code smell Num	[5]	[9]	[2]	[12]	[11]	[4]	[8]	[29]	[6]	[30]	Coverage
Year of publication	2021	2017	2020	2022	2017	2022	2022	2023	2022	2022	N/A
Code smell 1. Repeated computation of the same expression	○			○							0/10
Code smell 2. Extractable code chunks											0/10
Code smell 3. State Variable Refactoring			○								0/10
Code smell 4. Redundant operations with same effects			●	○	○		○	○			1/10
Code smell 5. Pre-computable operations on constants		○									0/10
Code smell 6. Deterministic conditional checks	●	●						●			3/10
Code smell 7. Conditional statements with simpler equivalents						●					1/10
Code smell 8. Replacing item-by-item iterated arrays by a map			●				●				2/10
Code smell 9. Repeated security checks across function calls								○			0/10
Code smell 10. Unnecessarily introducing variables											0/10
Code smell 11. Unnecessary overflow/underflow validation since Solidity 0.8.0											0/10
Code smell 12. Redundant memory array initialization											0/10
Code smell 13. Placement of require statements											0/10
Code smell 14. Avoid no-op writes to state variables			○							●	1/10
Code smell 15. Reordering conditional checks for short-circuiting			●	○				○			1/10
Code smell 16. Combinable events											0/10
Code smell 17. Add constant modifier for non-changing variables					●	●					2/10
Code smell 18. Function visibility			●	●	○	●	○				3/10
Code smell 19. Dead codes		●						●			2/10
Code smell 20. Using revert instead of require for error handling											0/10
Code smell 21. Minimization of event message string											0/10
Code smell 22. Replacing MUL/DIV of powers of 2 by SHL/SHR											0/10
Code smell 23. Struct variable reordering			○	○	○	●					1/10
Code smell 24. Loop invariant codes	●	●				●		●			4/10
Code smell 25. Avoid expensive operations inside loops	○	●					○	○		●	2/10
Code smell 26. Using bytes32 for string representation							●	●		○	2/10

Overall, six relevant papers [2], [4], [5], [8], [12], [29] are identified.

- 3) **STEP 3: Snowballing search.** To further capture any potentially overlooked papers, we adopt the snowballing search technique. This is done by taking two directions of search, including the papers that are cited by the papers we identified (i.e., backwards snowballing), as well as the papers that cited them (i.e., forwards snowballing). This step added four more relevant papers [6], [9], [11], [30] to our list.

Overall, our list of identified papers span the period from the year of 2017 up to 2023, with [9] being the earliest work that formally presented gas-wasting code smells. These papers found gas-wasting code smells from various sources, including manual observation, past coding experience, textbooks, online blog posts, discussion forums, and research papers. It is worth noting that, to our best knowledge, we are the first to integrate an LLM (i.e. GPT-4) into the pipeline of the identification of gas-wasting code smells.

The results of comparison to past papers are shown in Table III, where we show exactly which of our identified code smells are covered by which of the related papers. Note that there are two types of labels. First, there is "identical" (labeled by ●), which means the corresponding paper presented a code smell that is exactly the same as ours. Second, there is "tangentially related" (labeled by ○), which means the corresponding paper presented a code smell that is tangentially related to ours from a similar perspective, but not quite the same yet. As an example for the "tangentially related" case, our code smell 15 (i.e. reordering conditional checks for short-circuiting) proposes to reorder logical statements to place less likely conditions first to utilize the short-circuiting rule of the "and" operator, but two papers [12], [29] instead suggests to place less costly conditional statements first. These are both legitimate approaches that present similar perspectives, and their tradeoff is open for discussion. Overall, out of the 26 code smells we identified, 13 code smells are fresh (i.e. have not been covered at all in existing papers), which signifies the novelty of our identified code smells. In addition, it is worth noting that we have the largest set of gas-wasting code smells among related works, where largest set of code smells from past papers had 24 code smells, and on average the 10 papers proposed 9.4 code smells per paper.

Overview for RQ1: Overall, our LLM-facilitated identified set of code smells are the largest among existing related works, and most of our code smells (13 out of 26) were not presented in previous papers.

C. RQ2: Effectiveness: the amount of gas saved.

At this section, we aim to get a better understanding of how our identified code smells perform in terms of saving gas. In particular, we have compiled a report for all of the 26 code smells, where for each code smell, we present an explanation of it, an example contract extracted from our dataset of recently deployed contracts (some are from existing papers, as

explained in the note below) as well as its associated amount of saved gas, and an analysis of where the gas saving comes from. To our best knowledge, such level of detail is not seen in past works. Following the conventional practice of existing papers, we present a pair of optimized and unoptimized contract codes for each example in the report, where the optimized codes are what we would obtain after fixing a certain gas-wasting code smell on the unoptimized codes. Note that the original contract codes are simplified to reduce the complexity of presentation and to focus the spotlight on the gas-wasting code smells rather than other unrelated code segments. Overall, the full report is 47 pages long and is available at [55]. Upon paper acceptance, we intend to further publish all of our datasets and documents that we collected during our work. (As a note, 4 of the 26 presented code smells (Code smells 19, 23, 24, and 25) are discovered from the ablation phase without any input codes, so the code smells were not reported from real-world contracts. However, fortunately, these code smells are all covered in existing papers and we are able to find example contracts from these papers.) Due to space limitation, we are not able to include the full report here. Instead, as a demonstration of our results, we use case studies to present 4 of the 13 new code smells that we identified, and the case study is presented at Section V.

In particular, we adopted Remix [56] to measure the amount of saved gas, which is an integrated development environment (IDE) recommended by Ethereum [57]. In particular, we deployed each of our test contracts on a VM provided by the IDE that implements the Shanghai version [58], with a gas limit of 3000000. We use the debugger feature to run transactions to contracts, where each execution of a transaction returns information including the status, transaction hash, block hash, block number, from, to, gas, execution cost, etc. We extract the "execution cost" field as our measured gas cost and report them in our results. The results are shown in Table IV. It can be read from Table IV that for a few code smells (4 of the 26 code smells), their deployment cost might increase as a tradeoff for a lower running cost, or vice versa. Others (i.e. 21 out of the 26 code smells) have gas savings for both deployment and message call. Overall, the gas saving is averaged at 10.534 % for the deployment cost of the 26 contracts, and 21.528 % for the message call cost, which is a signification of the effectiveness of our identified code smells from the perspective of saving gas.

Overview for RQ2: On average, our LLM-facilitated identified set of code smells save about 10.534 % of the deployment cost and 21.528 % of the message call cost.

D. RQ3: Contribution of design components: an ablation study.

In this section, with the goal of better understanding the effectiveness of our design choices, we perform an ablation study. In particular, we selected 4 components to perform ablation on: few-shot examples, chain-of-thought prompting, self-inspection, and codes. Note that we remove codes for the

TABLE IV: Comparison of the gas cost of the unoptimized and optimized contracts, where $\mathcal{D}_i$ stands for deployment costs, $\mathcal{M}_i$ stands for message call costs, and $i \in \{u, o\}$ with u standing for the unoptimized version and o for the optimized one. $\mathcal{S}$ stands for the percentage of saved gas.

Code smell #	$\mathcal{D}_u$	$\mathcal{D}_o$	$\mathcal{S}_{\mathcal{D}}$	$\mathcal{M}_u$	$\mathcal{M}_o$	$\mathcal{S}_{\mathcal{M}}$
Code smell 1.	166,408	155,602	6.4936%	6,090	5,327	12.528%
Code smell 2.	500,672	433,810	13.354%	7,958	8,021	-0.791%
Code smell 3.	146,106	130,506	10.677%	12,982	3,109	76.051%
Code smell 4.	206,851	206,851	0.0%	1,847	1,831	0.8662%
Code smell 5.	234,675	223,263	4.8628%	2,028	1,732	14.595%
Code smell 6.	177,820	174,420	1.9120%	2,487	2,447	1.6083%
Code smell 7.	166,957	163,150	2.2802%	2,798	2,750	1.7155%
Code smell 8.	536,968	392,430	26.917%	99,428	94,711	4.7441%
Code smell 9.	350,386	306,143	12.626%	4,384	4,314	1.5967%
Code smell 10.	218,263	208,051	4.6787%	1,790	1,676	6.3687%
Code smell 11.	204,845	179,826	12.213%	13,553	12,953	4.4270%
Code smell 12.	93,541	43,293	53.717%	691	399	42.257%
Code smell 13.	352,233	352,233	0.0%	4,444	3,598	19.036%
Code smell 14.	92,541	94,141	-1.728%	3,001	2,910	3.0323%
Code smell 15.	280,518	280,518	0.0%	1,150	1,019	11.391%
Code smell 16.	142,990	116,565	18.480%	5,668	4,028	28.934%
Code smell 17.	724,419	768,627	-6.102%	34,092	28,588	16.144%
Code smell 18.	339,980	339,980	0.0%	25,712	5,812	77.395%
Code smell 19.	97,947	89,941	8.1738%	1,147	869	24.237%
Code smell 20.	197,220	150,171	23.856%	980	692	29.387%
Code smell 21.	116,565	78,529	32.630%	4,028	2,088	48.162%
Code smell 22.	111,759	85,335	23.643%	783	540	31.034%
Code smell 23.	207,451	207,451	0.0%	70,537	68,637	2.6936%
Code smell 24.	153,571	154,771	-0.781%	80,330	41,931	47.801%
Code smell 25.	109,559	110,159	-0.547%	81,417	59,644	26.742%
Code smell 26.	449,628	330,316	26.535%	11,341	8,191	27.775%
Average			10.534%			21.528%

purpose of probing the memory of GPT-4 [59], as this requires it to come up with gas-wasting code smells without looking at any codes.

In particular, we took the last 20 contracts from the code smell discovery phase (i.e. rounds 7 and 8) as our testing dataset of contracts. We then removed one of the aforementioned ablation components at a time, ran our code-smell-finding procedure on them, and collected statistics along the way. This was done for two rounds with 10 contracts each (i.e. the first round uses codes from round 7, and the second uses round 8 codes). The results are shown in Table V, where the top cell (i.e. the first 4 rows) represent the statistics from the first round, and middle cell represent the second round, and the bottom cell (i.e. the last 4 rows) represent the statistics for all the 20 contracts altogether.

It could be observed that the removal of CoT incurs either the second lowest (top and bottom cells) or the lowest (middle cell) pass rate ($\mathcal{P}_{pass}$), as well as the lowest number of passed categories ($\mathcal{N}_{c_{pass}}$). In addition, its entropy of verified code smells ($\mathcal{E}_{pass}$) is also noticeably low and thus more

biased towards certain categories, which means it tends to find more repeated categories instead of new ones. These metrics are discussed in more detail at Section III-D. Overall, the performance of the LLM largely drops due to the removal of CoT, consistently for the the first round, second round, and both rounds combined. Interestingly, the pass rate ($\mathcal{P}_{pass}$), the number of passed code smell categories ($\mathcal{N}_{c_{pass}}$), and the entropy of verified categories ($\mathcal{E}_{pass}$) are all the highest among all when we are probing the memory of GPT-4 (i.e. with input codes removed). This reflects that GPT-4 already has some internal knowledge of gas-wasting code smells, and was elicited by our experiment. It is worth noting that memory probing resulted in the lowest entropy of rejection reasons ($\mathcal{E}_{rej}$), which means that GPT-4 tends to make the same mistake more often. In particular, out of the 39 reported code smells that were rejected, 28 are categorized as being "Too superficial or common-sense to be a valid or interesting gas-wasting code smell," where one example is a code smell stating that "Accessing local variables is cheaper than accessing state variables." This means that memory probing resulted in too

TABLE V: Ablation results, where $\checkmark$ means the component is present, and $\times$ means the component is removed.

CoT	FSE	CODES	SI	$\mathcal{P}_{pass}$	$\mathcal{P}_{rej}$	$\mathcal{N}_{pass}$	$\mathcal{N}_{rep}$	$\mathcal{N}_{c_pass}$	$\mathcal{N}_{c_rej}$	$\mathcal{E}_{pass}$	$\mathcal{E}_{rej}$	$\mathcal{A}_{in_lms}$	$\mathcal{P}_{val}$
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	0.385	0.615	10	26	4	6	1.357	2.274	168.7	0.8
$\times$	$\checkmark$	$\checkmark$	$\checkmark$	0.235	0.765	8	34	3	6	1.061	2.378	167.7	0.8
$\checkmark$	$\times$	$\checkmark$	$\checkmark$	0.333	0.667	10	30	6	5	2.161	2.171	158.4	0.8
$\checkmark$	$\checkmark$	$\times$	$\checkmark$	0.471	0.529	16	34	7	2	2.35	0.65	22.0	1.0
$\checkmark$	$\checkmark$	$\checkmark$	$\times$	0.128	0.872	6	47	5	7	2.252	2.463	161.7	0.5
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	0.25	0.75	6	24	3	7	1.252	2.281	179.0	0.5
$\times$	$\checkmark$	$\checkmark$	$\checkmark$	0.143	0.857	5	35	1	8	0.0	2.506	178.0	0.5
$\checkmark$	$\times$	$\checkmark$	$\checkmark$	0.179	0.821	5	28	2	8	0.722	2.753	164.4	0.5
$\checkmark$	$\checkmark$	$\times$	$\checkmark$	0.355	0.645	11	31	6	6	2.369	1.679	22.0	0.5
$\checkmark$	$\checkmark$	$\checkmark$	$\times$	0.224	0.776	11	49	3	8	0.866	2.626	172.0	0.6
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	0.32	0.68	16	50	5	10	1.497	2.511	173.85	0.65
$\times$	$\checkmark$	$\checkmark$	$\checkmark$	0.188	0.812	13	69	3	8	0.773	2.618	172.85	0.65
$\checkmark$	$\times$	$\checkmark$	$\checkmark$	0.259	0.741	15	58	7	9	2.005	2.671	161.4	0.65
$\checkmark$	$\checkmark$	$\times$	$\checkmark$	0.415	0.585	27	65	10	6	2.619	1.37	22.0	0.75
$\checkmark$	$\checkmark$	$\checkmark$	$\times$	0.177	0.823	17	96	7	9	1.849	2.708	166.85	0.55

many reported code smells that are too common-sense to be regarded as a valid code smell. In the case of removing self-inspection, there is the highest number of reported code smells ($\mathcal{N}_{rep}$) but the lowest pass rate ($\mathcal{P}_{pass}$). Since the removal of SI reduces the workload of GPT-4 in terms of answering reflective questions, we hypothesize that the removal of self-reflecting requirements releases more attention resources, which in turn gets distributed to focus on reporting more potential code smells. We leave this hypothesis for future works to verify.

Overview for RQ3: Overall, the performance of GPT-4-facilitated code-smell-finding drops when FSE (few shot example), CoT (chain of thought prompting), or SI (self inspection) are removed, with the removal of CoT exhibiting the worst performance across multiple evaluation dimensions.

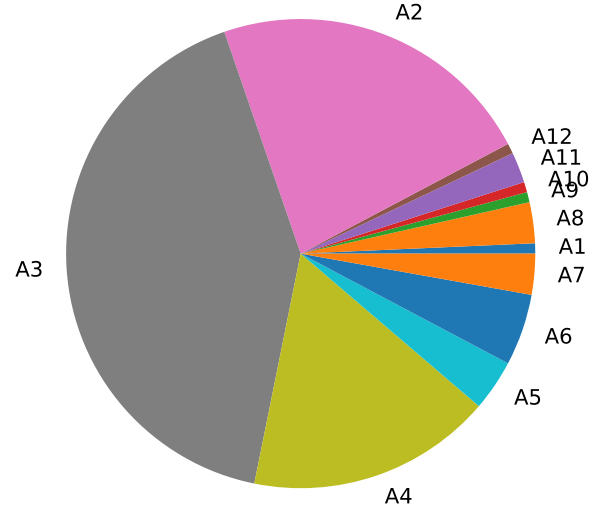


Fig. 3: The pie chart for the passed category.

E. RQ4: How GPT-4 failed.

At this section, we aim to present our findings of the categorization of code smell categories as well as GPT-4 failures. In particular, while we were performing verification over the reported code smells from GPT-4, we also categorized the reasons of why each of the failed code smells was rejected. The categorization procedure is described in more detail at Section III-C. In total, we kept track of 13 categories of reasons of failure, and we plot them in the form of a pie chart in Figure 3. We prepend each category by the letter 'A' to distinguish them from code smell categories. A full list is provided in our full report. Below is a list of the top 3 categories of why gpt-4 failed to report correct code smells:

- 1) A3. Based on non-existent or incorrect code information (e.g. claiming that a variable was used twice, while actually it is only once).
- 2) A2. The found code smell is conditioned on some not-so-likely assumptions (e.g. asking to move require statements to be validated offchain to save onchain gas)).
- 3) A4. Misunderstanding of the execution sequence of codes (e.g. misunderstanding that all defined functions are to be executed sequentially at once, or all branches of if-else statements are to be executed in order).

The issues 1 and 3 observed above correspond to the

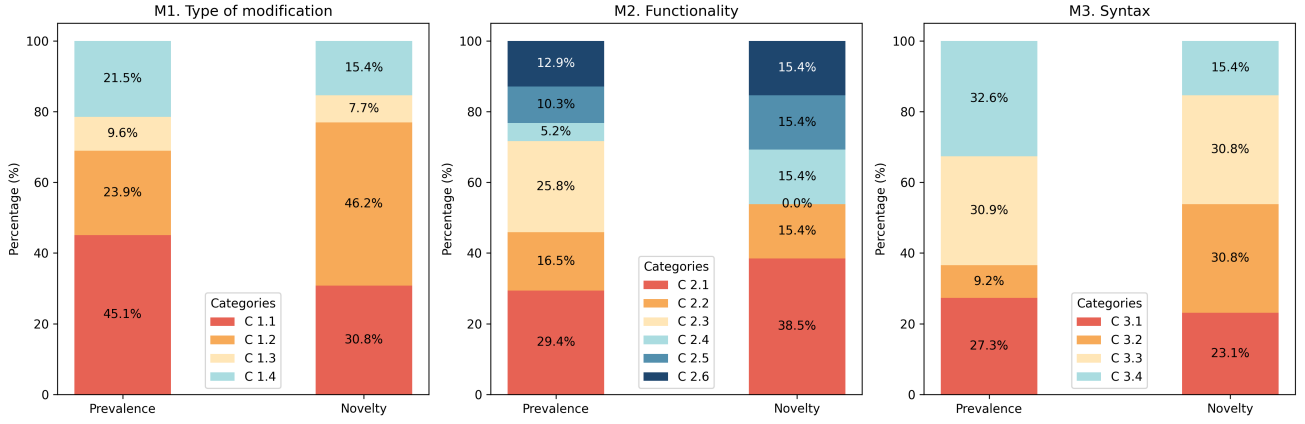


Fig. 4: Three types of meta-categorization.

problems of hallucination and being confidently wrong in its predictions as documented by the official GPT-4 report [16]. The issue 2 is more of an edge case where the reported code smells are somewhat reasonable to a certain extent but extremely difficult to be implemented or triggered. Overall, these 3 top categories account for $80.99\% = \frac{115}{142}$ of all the rejected code smells that were reported by GPT-4.

Overview for RQ4: Hallucination remains a major problem of GPT-4 in our experiment, which raises alarms for the issue of over-reliance on LLMs.

F. RQ5: Importance of code smells.

In this section, we analyze the importance of the identified code smells by organizing them into various meta-categories from three different perspectives. These perspectives encompass both intrinsic and extrinsic features of the code smells: intrinsic features relate to the syntax and functionality, while extrinsic features pertain to the types of actions required to address these code smells. The metrics adopted for assessing the importance of each category include *prevalence* and *novelty*, calculated as the percentage of contracts under each category and the percentage of newly discovered code smell types under each category, respectively. To avoid imbalanced comparison, given the variable number of code smells under each meta-category (e.g., some meta-category might contain more code smells and thus more contracts), we use the average number of contracts per code smell to compute the *prevalence* metric. We introduce each of the 3 types of meta-categorization below, and show the results at Figure 4:

1) *Meta-categorization 1 (M1): By the type of required modification:* This categorization is based on the type of treatment action required to address the code smell, reflecting the complexity of maintenance and potential impact on code optimization:

- 1) **Category 1.1: Deletion.** Targets redundancy-related code smells which require the deletion of certain code elements. These are inefficiencies related to unnecessary codes that don't add value and can be removed.

- 2) **Category 1.2: Replacement.** Involves replacing code elements with better alternatives. This category simplifies the code by directly replacing an element with a more efficient option.
- 3) **Category 1.3: Reordering.** Targets order-dependent code smells that require reordering code lines and making corresponding adjustments, which can optimize execution flow or resource usage.
- 4) **Category 1.4: Refactoring.** Requires more comprehensive code restructuring, which involves more complex operations than simple replacement or reordering.

2) *Meta-categorization 2 (M2): By functionality:* This categorization groups code smells based on the functionality they affect, which helps in understanding the operational impact of each code smell:

- 1) **Category 2.1: Control flow.** Includes code smells that affect how functions are called and how the execution flow is managed within the contract.
- 2) **Category 2.2: State and memory.** Relates to how the contract manages its state and uses memory.
- 3) **Category 2.3: Data Structure and Variable Type.** Targets the optimal use of data structures and data types, leading to more efficient data storage and retrieval.
- 4) **Category 2.4: Event.** Involves code smells related to the generation and handling of events and logs.
- 5) **Category 2.5: Arithmetic operations.** Involves optimizing logical and arithmetic operations to reduce computational overhead.
- 6) **Category 2.6: Error handling.** Involves code smells that are related to the security and correctness aspect of the contract.

3) *Meta-categorization 3 (M3): By syntax:* Finally, code smells are categorized by their syntactic characteristics, highlighting different levels of syntactic granularity:

- 1) **Category 3.1: Modifier.** Involves simple single-word changes such as modifiers and variable types.
- 2) **Category 3.2: Single Expression/Statement.** Optimization at the level of an individual line or expression.
- 3) **Category 3.3: Code Block.** Optimization across multiple statements, requiring a broader understanding of the code context.

- 4) **Category 3.4: Functions.** Changes at the function level affecting function declaration or utilization.

Overall, it can be observed from Figure 4 that for M1, direct replacements and redundancy elimination are the most prevalent interventions needed, indicating a significant potential for gas savings through straightforward alternatives. Meanwhile, for M2, control flow related issues are more prevalent, suggesting that these elements play a more critical role for performance and efficiency among our identified code smells. Furthermore, M3 presents a more balanced distribution of gas-wasting code smells across different syntactic constructs, with the exception of the "Single Expression/Statement" category. This suggests that the majority of identified issues are related to broader syntactic elements rather than isolated lines of code, which implies that optimization efforts should focus on more comprehensive syntactic modifications. From the perspective of novelty, the categories generally follow a similar trend to that of the prevalence. However, there is one exception; for M2, none of our identified code smells regarding "Data Structure and Variable Type" are novel, suggesting that these issues have been more extensively studied in existing works. Overall, this research question provides insights in terms of both the prevalence and novelty of our identified code smells, which could be utilized to better guide future research on the field of gas optimization.

Overview for RQ5: By categorizing the identified code smells from both intrinsic and extrinsic perspectives, the importance of code smells are characterized from multiple perspectives including type of required modification, functionality, and syntax.

G. RQ6: Extensibility of the proposed framework.

To answer this research question, we analyze and discuss the extensibility of our framework from three perspectives, including the extensibility to a different programming language, to new EVM features, and to a different LLM.

1) *Extensibility to other programming languages.*: To begin with, we explore the potential of extending our proposed framework to a different programming language. In particular, we choose Vyper [60], which is the second most popular programming language for Ethereum smart contracts after Solidity. Similar to Solidity, Vyper targets the Ethereum Virtual Machine (EVM) and compiles to EVM bytecode. One of the major differences between these two languages is syntax: Solidity's syntax closely resembles C++ or Java, while Vyper adopts a Python-like syntax. Additional distinctions include compiler optimization strategies, support of inline assembly, modifiers, class inheritance, etc. Figure 13 and Figure 14 illustrate two sample Vyper contracts. In order to test our framework with Vyper, we crawled the 500 most recently verified Vyper smart contracts from Etherscan [36]. We apply the same contract selection and code smell discovery procedures as used for Solidity, which are described in Section III-A, by only changing the mentioning of the programming language within the prompting instructions from Solidity to Vyper. As

a result, a total of 26 contracts were selected out of the 500 Vyper contracts that we crawled. In addition, 6 different types of code smells were identified, with one of them being new and having not been observed in our analysis of Solidity contracts. We present two example Vyper contracts, including the newly identified code smell, as a case study at Section V-E². Overall, the identification of the above code smells, including a new one, indicates that our framework can be effectively extended to other programming languages with minimal adjustments. This finding is significant because, to our best knowledge, there are no existing studies that focus on identifying gas-wasting code smells specifically in the Vyper programming language.

2) *Extensibility to new EVM features.*: As an additional step, we explore the potential of extending our proposed framework to accommodate newly proposed features in the EVM. We focus on the implications of Ethereum Improvement Proposals (EIPs), via which changes to the EVM are officially proposed, reviewed, and implemented. Specifically, we select the EIP-7609 [61], which is one of the newest EIPs that proposes changes to the EVM gas cost and opcode system³. In particular, it is closely pertinent to the newest Cancun update⁴ that was recently introduced to Ethereum, in which *transient storage* [62] was proposed as a new storage location for transaction-wide state management that does not require disk access, which was designed for more efficient inter-frame communication. Transient storage functions similarly to persistent storage as a key-value store, but exists only for the duration of a transaction. EIP-7609 specifically proposes the following gas costs associated with transient storage operations:

- 1) TSTORE: 5 gas
- 2) TLOAD: 8 gas plus *expansion_cost*, where *expansion_cost* is 0 if the queried key is already in transient storage, and is computed as the number of keys already in transient storage otherwise (i.e., superlinear pricing).

To assess the adaptability of our framework to these changes, we apply the same code smell discovery procedures and smart contracts as used for the regular version of EVM, as detailed at the step 4 of Section III-A, with the only difference being an added prompting instruction to hypothesize the implementation of the EIP-7609. The goal is to analyze the impact of EIP-7609 on smart contracts. Overall, the results show that 22 % of the investigated contracts exhibit issues specific to the introduction of transient storage and its pricing mechanism, which can be divided into 2 types of novel code smells:

- 1) **1. Intra-function:** Opportunities for caching one storage variable across multiple locations within a function.
- 2) **2. Inter-function:** Multiple functions could utilize transient storage within the same transaction, enabling cross-function caching.

²A complete list of the code smells and their associated contracts is available at our public repo as introduced in Section VIII

³Proposed as a draft in February, 2024

⁴Introduced in March, 2024

TABLE VI: Statistics of each round of Claude-3-facilitated gas-finding.

Round_num	$\mathcal{P}_{pass}$	$\mathcal{P}_{rej}$	$\mathcal{N}_{pass}$	$\mathcal{N}_{rep}$	$\mathcal{N}_{c_pass}$	$\mathcal{N}_{c_rej}$	$\mathcal{E}_{pass}$	$\mathcal{E}_{rej}$	$\mathcal{A}_{in_Ins}$	$\mathcal{P}_{val}$
1	0.2	0.8	6	30	3	6	1.252	1.884	169.9	0.4
2	0.276	0.724	8	29	6	6	2.5	2.131	152.7	0.6
3	0.273	0.727	9	33	5	10	2.197	2.635	191.2	0.6
4	0.276	0.724	8	29	4	5	1.549	1.625	168.7	0.7
5	0.115	0.885	3	26	3	7	1.585	2.282	179.0	0.3

We present one example contract, as a case study in Section V-F, to further illustrate both types of optimization opportunities. Overall, these code smells suggest that if the newly proposed EIP-7609 were to be implemented, it could lead to a significant shift in programming paradigms by promoting extensive usage of transient storage. Note that without EIP-7609, there would not be such a strong incentive, since the warm storage accesses and writes would cost the same amount of gas as their counterpart transient storage operations. This serves as a critical implication of the effect of this EIP, showing that our proposed framework could be extended to accommodate newly proposed EVM features and at the same time discover some potential future impacts brought by these EIPs.

3) *Extensibility to other LLMs.*: At this section, in addition to the extensibility to a different programming language and novel EVM features as discussed above, we explore the potential of extending our proposed framework to a different LLM. Specifically, we choose Claude-3 for our investigation, which is a newly released LLM⁵ by Anthropic [63]. We adopt the Claude-3-Opus version for our experiments, which is reported by Anthropic to exhibit superior performance across a diverse range of tasks [64]. To assess the adaptability of our framework to Claude-3, we apply the same code smell discovery procedures and smart contracts as used for GPT-4, as detailed at the step 4 of Section III-A. The resulting statistics are shown at Table VI. It can be observed that Claude-3, when tested under the same settings, consistently demonstrates similar but slightly lower performance compared to GPT-4 across multiple metrics such as the pass rate ($\mathcal{P}_{pass}$) and number of discovered code smells ($\mathcal{N}_{pass}$); these metrics are discussed in more detail at Section III-D. Nonetheless, Claude-3 is still effective in terms of discovering a variety of code smells. These findings suggest that our framework could be extended to a different LLM (i.e., Claude-3), without necessitating modifications.

Overview for RQ6: Our proposed framework exhibits promises in terms of being extensible to different programming languages, novel EVM features, and other LLMs.

V. CASE STUDIES

In this section, we present four example contracts as a case study to demonstrate our identified gas-wasting code smells.

⁵Released on Mar 4, 2024.

A. Case study 1: State variable refactoring.

1) *Code smell explanation.*: This code smell seeks to specifically rearrange the layout of the state variables in storage to a more compact form by changing the type of variables (e.g. uint256 to uint32), while ensuring that the changed variable type is still compatible with the task at hand.

2) *Example and analysis.*: To further illustrate this code smell, we take as an example the contract *PumpNft*, which is deployed at 0xE3C7b06e06EAc93C9E3B11ea315C838A90CFB4ab. The unoptimized codes are shown in Figure 5, and the optimized version is shown in Figure 6. From the unoptimized codes, it can be observed that the variables *gStartsAt*, *gEndsAt*, *fcfsStartsAt*, *fcfsEndsAt*, *publicStartsAt*, *publicEndsAt* are all state variables of the *PumpNft* contract, and are all declared as **uint256**. However, since these variables are used as Unix timestamps to be compared to block timestamps, 256 bits would be overly large for this purpose. As a reference, existing mature systems like MariaDB [65] and MongoDB [66] use 4 bytes to store timestamps. Therefore, it is not only safe but also enough to reduce the uint256 data type of the aforementioned state variables to uint32 in this example. Our experiment shows that this contributes to a significant saving of gas (i.e. a saving of 10.677% deployment gas and 76.051% message call gas).

Next we explain how gas gets saved. To begin with, note that operations on storage are quite expensive [67]. The cost of SLOAD is 2100 for cold accesses (the executing contract has not touched the target storage slot yet), and 100 for warm accesses. In addition, the cost of SSTORE is much more complicated, and it depends on factors such as if the access is cold (extra 2100 gas), if it is a no-op that stores the same value as what's already in the target slot (extra 100 gas), if it overwrites a clean slot that started with a zero value (extra 20000 gas), etc. To make the numbers more intuitive, note that the gas costs of a LT (less than) logical comparison is 3 gas, MOD is 5, MSTORE (storing a value in memory) without memory expansion is 3, and a logical jump is 8 [68].

Now we move on to discuss the gas cost of the codes. In particular, the gas saving comes from turning sequential cold accesses into repeated warm accesses to the same address. To be more specific, in the *currentStage* function of the unoptimized contract, since the state variables are declared as uint256, each would occupy a different storage slot. Because of this, during execution, the six accesses to each of the state variables are cold and would cost $12600 = 2100 \times 6$ gas in total. On the other hand, in the optimized version, the six

```

1 pragma solidity ^0.8.0;
2
3 contract PumpNft {
4     uint256 public gStartsAt;
5     uint256 public gEndsAt;
6     uint256 public fcfsStartsAt;
7     uint256 public fcfsEndsAt;
8
9     constructor(
10         uint256 _gStartsAt,
11         uint256 _gEndsAt,
12         uint256 _fcfsStartsAt,
13         uint256 _fcfsEndsAt
14     ) {
15         gStartsAt = _gStartsAt;
16         gEndsAt = _gEndsAt;
17         fcfsStartsAt = _fcfsStartsAt;
18         fcfsEndsAt = _fcfsEndsAt;
19     }
20
21     function currentStage() public returns (uint256
22         stage) {
23         if (block.timestamp >= gStartsAt && block.
24             timestamp <= gEndsAt) {
25             return 1;
26         }
27
28         if (block.timestamp >= fcfsStartsAt && block.
29             timestamp <= fcfsEndsAt) {
30             return 2;
31         }
32
33         return 0;
34     }
35 }

```

Fig. 5: Case study 1, unoptimized version.

```

1 pragma solidity ^0.8.0;
2
3 contract PumpNft {
4     uint32 public gStartsAt;
5     uint32 public gEndsAt;
6     uint32 public fcfsStartsAt;
7     uint32 public fcfsEndsAt;
8
9     constructor(
10         uint32 _gStartsAt,
11         uint32 _gEndsAt,
12         uint32 _fcfsStartsAt,
13         uint32 _fcfsEndsAt
14     ) {
15         gStartsAt = _gStartsAt;
16         gEndsAt = _gEndsAt;
17         fcfsStartsAt = _fcfsStartsAt;
18         fcfsEndsAt = _fcfsEndsAt;
19     }
20
21     function currentStage() public returns (uint256
22         stage) {
23         if (block.timestamp >= gStartsAt && block.
24             timestamp <= gEndsAt) {
25             return 1;
26         }
27
28         if (block.timestamp >= fcfsStartsAt && block.
29             timestamp <= fcfsEndsAt) {
30             return 2;
31         }
32
33         return 0;
34     }
35 }

```

Fig. 6: Case study 1, optimized version.

```

1 pragma solidity >=0.7.6;
2
3 contract LEGO {
4     event logOnChain(string info);
5
6     function loggingOnChain() public {
7         emit logOnChain("pKeuLv");
8         emit logOnChain("IOpBBS");
9         emit logOnChain("pOmZqnWO");
10        uint256 e = 87;
11        uint256 p = (56 % 73) % 56;
12        uint256 rzzlcjlf = 76 % 94;
13        uint256 joblgy = (77 % 86) % 41;
14        uint256 tfsleihel = 20 % 17;
15        if (
16            e == 56 * 71 &&
17            p == ((36 * 4) % 39) % 57 &&
18            rzzlcjlf == 95 + 56 + 85 * 80 &&
19            joblgy == 20 % 27 &&
20            tfsleihel == (72 % 3) * 91
21        ) return;
22    }
23 }

```

Fig. 7: Case study 2, unoptimized version.

```

1 pragma solidity >=0.7.6;
2
3 contract LEGO {
4     event logOnChain(string, string, string);
5
6     function loggingOnChain() public {
7         emit logOnChain("pKeuLv", "IOpBBS", "pOmZqnWO")
8         ;
9         uint256 e = 87;
10        uint256 p = (56 % 73) % 56;
11        uint256 rzzlcjlf = 76 % 94;
12        uint256 joblgy = (77 % 86) % 41;
13        uint256 tfsleihel = 20 % 17;
14        if (
15            e == 56 * 71 &&
16            p == ((36 * 4) % 39) % 57 &&
17            rzzlcjlf == 95 + 56 + 85 * 80 &&
18            joblgy == 20 % 27 &&
19            tfsleihel == (72 % 3) * 91
20        ) return;
21    }
22 }

```

Fig. 8: Case study 2, optimized version.

state variables are packed into the same storage slot since each is of a small size of 4 bytes and they altogether could fit in one slot. This means that only the first access to one of the state variables is cold (i.e. access to $gStartsAt$), and all subsequent ones are warm. This way, the cost gets reduced to $2600 = 2100 + 100 \times 5$ gas, which is much lower than the unoptimized version.

B. Case study 2: Combinable events.

1) *Code smell explanation.*: Consider whether all events are absolutely necessary. If they are, consider whether they

can be consolidated or whether the amount of data included can be reduced.

2) *Example and analysis.*: We take as an example the loggingOnChain function of the LEGO contract deployed at 0x111ACf72AA4A1fdA8500ED9f1Ba3F2374c02a21e. The unoptimized codes are shown in Figure 7, and the optimized version is shown in Figure 8.

To begin with, it is worth noting that the cost of emitting events consists of 3 components [27], as shown in Equation 6.

```

1 pragma solidity ^0.8.0;
2
3 contract UNCX_ProofOfReservesV2_UniV3 {
4     event onRemoveFee(bytes32);
5     function contains (bytes32) internal returns (bool)
6     {
7         return true;
8     }
9
10    function remove (bytes32) internal returns (bool) {
11        return true;
12    }
13
14    function removeFee (string memory _name) external {
15        bytes32 nameHash = keccak256(abi.encodePacked(
16            _name));
17        require(nameHash != keccak256(abi.encodePacked(
18            "DEFAULT")), "DEFAULT");
19        require(contains(nameHash));
20        remove(nameHash);
21        emit onRemoveFee(nameHash);
22    }
23 }

```

Fig. 9: Case study 3, unoptimized version.

```

1 pragma solidity ^0.8.0;
2
3 contract UNCX_ProofOfReservesV2_UniV3 {
4     event onRemoveFee(bytes32);
5     function contains (bytes32) internal returns (bool)
6     {
7         return true;
8     }
9
10    function remove (bytes32) internal returns (bool) {
11        return true;
12    }
13
14    function removeFee (string memory _name) external {
15        bytes32 nameHash = keccak256(abi.encodePacked(
16            _name));
17        require(nameHash != 0
18            x9F28225C7D0ACE67FA2516BD7725F3, "DEFAULT"
19        );
20        require(contains(nameHash));
21        remove(nameHash);
22        emit onRemoveFee(nameHash);
23    }
24 }

```

Fig. 10: Case study 3, optimized version.

$$G_{event} = G_{log} + G_{logdata} \times \mu_s[1] + n \times G_{logtopic} + C_{mem} \quad (6)$$

where $G_{log} = 375$ gas, $G_{logdata} = 8$ gas, $G_{logtopic} = 375$ gas, n is the number of topics, and C_{mem} is the memory expansion cost. In particular, it means 375 gas is paid for each emitted event, 8 gas is paid for each byte of the data to log, and 375 gas is paid for each of the included topics, up to 4 topics.

In the unoptimized example shown in Figure 7, 3 separate events (at lines 7, 8, and 9) are continuously emitted. These events each costs 1,518 gas, totaling to 4554 gas. It is worth noting that each event has an individual charge of 375 gas for a LOG1 operation. In addition, since each event is not anonymous and no arguments are indexed (i.e. no "indexed" keywords specified), there will be only one topic, which is just the hashcode of the event signature, and thus there is a 375 gas cost for each event in terms of the topic cost.

On the other hand, in the optimized codes, we change the event declaration at line 4 to accept 3 string, and combine lines 7, 8, and 9 into one event emission (i.e. *emit logOnChain("pKeuLv", "IOpBBS", "pOmZqnWO");*). This way, we get to reduce the gas cost down to 3,054 gas, which is a saving of 1,500 gas (32.94 %). This corresponds to saving the $750 = 2 \times 375$ gas of two LOG1 operations, as well as the $750 = 2 \times 375$ gas of two topics, since they are all combined into one line now.

C. Case study 3: Pre-computable operations on constants.

1) *Code smell explanation.*: This code smell occurs when there are operations that are performed on constants or variables whose values could be inferred without being compiled. This code smell wastes gas because such computations could be carried out before deploying the contract, which saves the gas consumption during a transaction call.

2) *Example and analysis.*: To further illustrate this code smell, we take as an example the contract *UNCX_ProofOfReservesV2_UniV3*, which is deployed at *0x7f5C649856F900d15C83741f45AE46f5C6858234*. The unoptimized codes are shown in Figure 9, and the optimized version is shown in Figure 10.

From the unoptimized codes, it can be observed that there is a check of whether the variable *nameHash* (i.e. the hash of the input string *_name*) is the same as the hash of the string "DEFAULT". In other words, it is checking if the input variable *_name* has the same hashcode as the string "DEFAULT". Note that each time the comparison is done, the string "DEFAULT" is first encoded and then hashed; this is highly repetitive since the hashcode could be computed offline and placed in the codes directly. To save gas, we directly replace the codes *keccak256(abi.encodePacked("DEFAULT"))* by the corresponding hashcode of "DEFAULT" (i.e. *0x9F28225C7D0ACE67FA2516B...*, extra bytes omitted). This optimization contributes to a saving of 4.86 % deployment gas cost and 14.60 % message call gas cost.

One might argue that this constitutes a compromise to code readability, since replacing codes by hashcodes loses self-explanability. However, a line of comment could be readily added to elaborate the purpose of this line of code, which still maximally retains the clarity without compromising gas costs.

D. Case study 4: Repeated security checks across function calls.

1) *Code smell explanation.*: The code smell of repeated security checks can be observed in function calls. Typically, the calling function initiates a security check, immediately followed by invoking another function that performs the same series of checks. This results in unnecessary redundancy. In these instances, the security check from the caller function could be eliminated to streamline the process.

2) *Example and analysis.*: To further illustrate this code smell, we take as an example the function the

```

1 pragma solidity ^0.8.0;
2
3 contract OperaBaseTokenTaxed{
4     mapping(address => mapping(address => uint256))
5         _allowances;
6     uint256 public _totalSupply;
7     mapping(address => uint256) _balances;
8
9     function transferFrom(
10         address sender,
11         address recipient,
12         uint256 amount
13     ) external returns (bool) {
14         require(sender != address(0), "ERC20: transfer
15             from the zero address");
16         require(recipient != address(0), "ERC20:
17             transfer to the zero address");
18
19         return _transferFrom(sender, recipient, amount)
20             ;
21     }
22
23     function _transferFrom(
24         address sender,
25         address recipient,
26         uint256 amount
27     ) internal returns (bool) {
28         require(sender != address(0), "ERC20: transfer
29             from the zero address");
30         require(recipient != address(0), "ERC20:
31             transfer to the zero address");
32
33         _balances[sender] = _balances[sender] - amount;
34
35         _balances[recipient] = _balances[recipient] +
36             amount;
37
38         return true;
39     }
40 }

```

Fig. 11: Case study 4, unoptimized version.

contract *OperaBaseTokenTaxed*, which is deployed at *0x0586638503CCaA365cD8a1338f3b84C54BAe65B3*.

Figure 11 and Figure 12 respectively display the unoptimized and optimized codes.

In the unoptimized contract, the *transferFrom* function conducts two security checks, represented by "require" statements. These checks are then repeated in the immediately following invocation of the *_transferFrom* function, unnecessarily resulting in redundancy. To optimize this, we eliminated the checks within the *transferFrom* function.

It's important to note that we didn't remove the checks from the *_transferFrom* function, as we can't be certain all callers of this function will consistently perform the necessary security checks. However, we have confidence in removing checks from the *transferFrom* function given that it consistently calls *_transferFrom*.

E. Case study 5: Vyper example contracts.

This case study demonstrates two example contracts for Section IV-G1.

1) *Contract 1*: This example demonstrates the Vyper smart contract deployed at *0xa53f49f8d6972884c53E2b4F3a5F4674A00bFC6d*. The codes are shown at Figure 13. This is a contract that implements a token system, with reference to several Ethereum token standards. In particular, there is a notable

```

1 pragma solidity ^0.8.0;
2
3 contract OperaBaseTokenTaxed{
4     mapping(address => mapping(address => uint256))
5         _allowances;
6     uint256 public _totalSupply;
7     mapping(address => uint256) _balances;
8
9     function transferFrom(
10         address sender,
11         address recipient,
12         uint256 amount
13     ) external returns (bool) {
14         return _transferFrom(sender, recipient, amount)
15             ;
16     }
17
18     function _transferFrom(
19         address sender,
20         address recipient,
21         uint256 amount
22     ) internal returns (bool) {
23         require(sender != address(0), "ERC20: transfer
24             from the zero address");
25         require(recipient != address(0), "ERC20:
26             transfer to the zero address");
27
28         _balances[sender] = _balances[sender] - amount;
29
30         _balances[recipient] = _balances[recipient] +
31             amount;
32
33         return true;
34     }
35 }

```

Fig. 12: Case study 4, optimized version.

```

1 @external
2 def _transferFrom(owner: address, receiver: address,
3     tokenId: uint256, sender: address):
4     # Check requirements
5     assert self._isApprovedOrOwner(sender, tokenId)
6     assert receiver != empty(address)
7     assert owner != empty(address)
8     assert self.idToOwner[tokenId] == owner
9     # Reset approvals, if any
10    if self.idToApprovals[tokenId] != empty(address):
11        self.idToApprovals[tokenId] = empty(address)
12    # Change the owner
13    self.idToOwner[tokenId] = receiver
14    _amt : uint256 = self.amt[tokenId]
15    # Change count tracking
16    self.balanceOf[owner] -= 1
17    self.brcBalanceOf[owner] -= _amt
18    # Add count of token to address
19    self.balanceOf[receiver] += 1
20    self.brcBalanceOf[owner] += _amt
21    # Log the transfer
22    log Transfer(owner, receiver, tokenId)

```

Fig. 13: Vyper example contract 1.

inefficiency concerning the management of balance updates. Specifically, at line 17, the balance of the *owner* is subtracted by the token amount *_amt* corresponding to *tokenId*, and then again added back to the *owner*'s balance at line 20. This ultimately results in a net change of zero, leading to redundant and expensive state updates, thereby incurring unnecessary gas costs. This constitutes an example of the

```

1 @external
2 @nonreentrant('lock')
3 def create_from_pool(
4     borrowed_token: address,
5     collateral_token: address,
6     A: uint256,
7     fee: uint256,
8     loan_discount: uint256,
9     liquidation_discount: uint256,
10    pool: address,
11    name: String[64],
12    min_borrow_rate: uint256 = 0,
13    max_borrow_rate: uint256 = 0
14 ) -> Vault:
15     # Find coins in the pool
16     borrowed_ix: uint256 = 100
17     collateral_ix: uint256 = 100
18     N: uint256 = 0
19     for i in range(10):
20         success: bool = False
21         res: Bytes[32] = empty(Bytes[32])
22         success, res = raw_call(
23             pool,
24             _abi_encode(i, method_id=method_id("coins(
25                 uint256)")),
26             max_outsize=32, is_static_call=True,
27             revert_on_failure=False)
28         coin: address = convert(res, address)
29         if not success or coin == empty(address):
30             break
31         N += 1
32         if coin == borrowed_token:
33             borrowed_ix = i
34         elif coin == collateral_token:
35             collateral_ix = i

```

Fig. 14: Vyper example contract 2.

Code Smell 4, *Redundant operations with same effects.*

2) *Contract 2*: This example demonstrates the Vyper smart contract deployed at `0xc67a44D958eeF0ff316C3a7c9E14FB96f6DedAA3`. The codes are shown at Figure 14. This contract implements a comprehensive system designed for creating and managing lending vaults. In particular, there is an inefficiency concerning the loop used for indexing tokens. This loop iterates up to a fixed number of times to find the indices of *borrowed_token* and *collateral_token* in a pool. However, even after a match is found, the loop does not terminate and instead continues to execute all the pre-specified number of iterations, regardless of whether the tokens have already been located early in the loop. To fix this inefficiency, the implementation should be modified to include a conditional break within the loop, to exit early if the token indices have been identified.

F. Case study 6: Transient storage in EIP-7609.

This case study is aligned with the discussions in Section IV-G2, and demonstrates the practical implications of EIP-7609 through an example contract. The example smart contract that we present is deployed at `0xcb1608a9c7E21e2AC5Ec44c63973C4a5A6BE7482`, and the codes are shown at Figure 15. The displayed contract *NLINK* extends the ERC20 token standard to include token swapping functionalities. In particular, it can be observed that the *swapTokens* function first calls *canSwapTokens* to determine the feasibility of a swap; if confirmed, it proceeds to call *swapBack* to execute the swap. Noticeably, there are multiple

```

1 contract NLINK is ERC20, Ownable {
2     function swapTokens(
3         address _account,
4         uint256 _amount,
5         uint256 _deadline
6     ) public {
7         address token = address(this);
8         require(swapTokensAtAmount <= balanceOf(token))
9         ;
10        if (canSwapTokens(_account, _amount, _deadline)
11            ) {
12            swapBack();
13        }
14    }
15
16    function canSwapTokens(
17        address _account,
18        uint256 _amount,
19        uint256 _deadline
20    ) internal returns (bool) {
21        // Some irrelevant lines omitted for brevity
22        bool moreThanSwappingTokens = balanceOf(self)
23        >= tokensForSwapping;
24        bool hasSwappingTokens = tokensForSwapping > 0;
25        if (hasSwappingTokens && moreThanSwappingTokens
26            ) {
27            _swap(swapper, tokensForSwapping);
28        }
29        tokensForSwapping = 0;
30        result = true;
31        return result;
32    }
33
34    function swapBack() private {
35        if (tokensForSwapping > 0 && balanceOf(address(
36            this)) >= tokensForSwapping) {
37            _swap(address(this), tokensForSwapping);
38        }
39        tokensForSwapping = 0;
40        // Some irrelevant lines omitted for brevity
41        tokensForSwapping = 0;
42        // Some irrelevant lines omitted for brevity
43    }
44 }

```

Fig. 15: New EVM feature example contract.

interactions with the uint256 state variable *tokensForSwapping* in the form of reads and writes. If the features of the EIP-7609, which is discussed in detail at Section IV-G2, were to be implemented, it would encourage gas consumption-aware developers to utilize the less costly transient storage for caching frequently accessed data like the *tokensForSwapping* variable. This way, the contract could significantly reduce the gas costs associated with repeated state variable access within a transaction. This optimization is not only beneficial within individual functions (*intra-function optimization*) but also across function invocations (*inter-function optimization*), due to the much lower gas cost associated with transient storage proposed by EIP-7609. Overall, this example is an illustration of how the newly proposed EIP could potentially shift programming paradigms by encouraging extensive usage of transient storage.

VI. DISCUSSION

A. Internal Validity.

Threats to internal validity mainly lie in the comprehensiveness of the identified code smells. First, there are false negatives. The inherent randomness in the output of GPT-4 means that taking only a single run on each chunk of codes could potentially overlook other code smells that exist within

the target codes, thus introducing the risk of false negatives. Some strategies, such as employing multiple runs (e.g., the pass@k metric [32], [69]), have been adopted to mitigate this issue. In addition, some code components like imports and some comments were not included in our constructed input code blocks, which was designed to conserve space. However, as prior research [70] pointed out, these components could significantly impact the performance of ChatGPT in tasks such as vulnerability detection. Thus, these could also introduce complications that might affect the performance of our framework. Furthermore, the employed filtering strategy at the step 2 and 3 of Section III-A could potentially filter out some contracts that contain novel gas-wasting code smells, which is a tradeoff by design, with the goal of placing the focus on a set of contracts that are more likely to contain varied and less repetitive components. In our investigation, we might have missed some code smells when applying GPT-4 due to the aforementioned reasons. However, we contend that this does not pose a threat as our objective is not to identify an exhaustive and complete list of all code smells within the examined codes. Instead, we aim to uncover a novel and diverse array of code smells, implying that incompleteness in the identified set of code smells does not reduce the validity of our results. In addition to false negatives, the code smells reported by GPT-4 may contain false positives due to its hallucination and faithfulness issues [16]–[20]. To minimize this, we have manually audited each of the reported code smells, extracted example codes for each code smell, and conducted a thorough analysis of the corresponding gas-saving mechanisms. This approach significantly reduces the likelihood of false positives that could arise due to GPT-4’s weaknesses.

B. External Validity.

At a high level, our methodology of finding defects (in our case, gas-wasting code smells) in programming languages could be viewed as a fusion of prompt engineering and injection of domain-specific knowledge (e.g. EVM and the Solidity programming language). In particular, for the former component, we take into considerations the weaknesses and strengths of GPT-4 when designing our input prompt framework. Along with the associated insights we gleaned, we primarily target the usage of LLMs, which is largely task-agnostic. This implies that our derived insights on this aspect offer valuable empirical lessons that are potentially transferable to other applications of LLMs. In addition, our proposed framework could be readily applied to other Solidity code snippets not included in our crawled dataset. For instance, it can be used to detect gas-related issues during the development process of new Solidity code bases by running the pipeline on each of the code files in a project. Overall, these two aspects signify the applicability and transferability of our framework.

C. Toy Contracts.

Toy contracts are primarily deployed for experimentation or trial purposes. They are characterized by being less actively used and are more likely to involve sub-optimal state due to

their ongoing development phase. Therefore, it is crucial to consider these contracts when analyzing the results of our study. Specifically, we follow the method of an existing work [71] to identify toy contracts, by checking the satisfaction of either of the following two criteria:

- 1) **Criteria 1. Activity level:** the contract received at least 29 transactions.
- 2) **Criteria 2. Currency influx:** the contract received at least 100 USD worth of ETH.⁶

Overall, using these criteria helps to filter out contracts that are more actively used based on both engagement and economic activity. By applying these criteria to the contracts in our study, we discovered that out of the 26 code smells we identified, 9 are unique to toy contracts, 7 are unique to non-toy contracts, and 10 are common to both. Furthermore, from the 13 novel code smells identified in Section IV-B, 5 are solely from toy contracts, 3 from non-toy contracts, and 5 were found in both. This distribution demonstrates a balanced representation of discovered code smells across both toy and non-toy contracts, highlighting that our proposed framework is capable of aiding developers to detect gas-wasting code smells effectively across various stages of contract development, covering both experimental and more established contracts.

D. Estimating improvement of productivity.

In this section, we discuss the performance of our proposed framework in terms of productivity enhancement. Specifically, we measured the time it took us to complete the verification of the reported code smells for each of the 50 contracts we verified at Section IV-G3. The time taken to read the codes and verify the reported code smells was averaged at 3.096 minutes per contract. This is a substantial reduction compared to the typical procedures required for manual code reading and code smell discovery. Manual approaches to code reading and comprehension, as evidenced by existing research, generally require significantly more time. For instance, one work [72] measured the time taken by 20 participants to read and comprehend realistic, non-trivial code segments, and reported an average time consumption of 26.0 minutes for 53 lines of codes. As a rough comparison, our approach demonstrates an efficiency gain by reducing the required effort by at least 88.09% ($\frac{26-3.096}{26} \times 100\%$). Another study [73] conducted a set of similar experiments and reported an average time consumption of 32.2 minutes for comprehending 125 lines of code, suggesting a 90.39 % reduction in effort (i.e., $\frac{32.2-3.096}{32.2} \times 100\%$).

It is worth noting that these comparisons are conservative for two reasons. First, the cited studies did not involve the additional task of discovering and verifying gas-wasting code smells, which could account for extra time consumption. Second, our inputs consist of more than 100 lines, which is more than twice of the size of codes adopted in the first referred study [72]. Thus, the complete pipeline of code reading, comprehending, and code smell identification would consume even more time than the referred work, and thereby

⁶The Ether price we used is 3,505.85 USD per ETH, taken from 1 Apr 2024

potentially a higher reduction in manual efforts. Overall, these comparisons show that leveraging a LLM-based framework for code smell identification in our study can significantly reduce the required manual efforts on code reading and analysis, highlighting the productivity improvement offered by our proposed framework.

E. Future works.

In this paper, the prompt framework introduced at Section III-B puts a combination of multiple tuning techniques into a modular design, keeping the prompt manageable and updatable. Recent research has suggested additional novel prompt tuning techniques. One survey study [74] proposes to divide prompt tuning techniques into multiple categories including "New Tasks Without Extensive Training," "Reasoning and Logic," "Reduce Hallucination," among others. Although our work exhibits a good coverage of these perspectives, this field continues to evolve rapidly with the introduction of novel techniques such as thread of thought (ThoT) prompting [75], graph-of-thoughts (GoT) prompting [76], tree-of-thoughts (ToT) prompting [77], [78], chain-of-note (CoN) prompting [79], and so on. These recently emerging additional techniques might constitute better alternatives that could be experimented with. We encourage future works to experiment with these techniques to see if effectiveness could be improved. The results from our work can serve as baseline statistics for future comparative studies, providing a reference point for evaluating the enhancements brought about by new prompting strategies.

Another potential area of improvement is the incorporation of call information. In our experiments, the call information was shown to be helpful in identifying certain code smells. For instance, code smell 9, which involves repeated security checks across function calls, was discovered in multiple input code blocks containing both the caller and callee functions. This indicates that our current approach already captures a certain level of call information. However, alternative methods to construct the input code block, such as leveraging a call graph, could potentially further enhance the effectiveness in identifying cross-function gas-wasting code smells by placing a focus specifically on call information. A call graph, where each node represents a function and each edge denotes a function call relationship, could be traversed using a depth-first search (DFS) algorithm. This method could organize the code block by control flow determined by function calls, potentially increasing the likelihood of identifying cross-function gas-wasting code smells. We encourage future works to experiment with this approach to see if effectiveness could be improved.

VII. RELATED WORK

Gas Optimization for Solidity. Ever since the emergence of the first paper [9] that proposed gas-saving code smells, this field has seen many works that attempt to enrich the toolkit of gas optimization. Overall, the gas optimization techniques could be divided into three categories: super-optimization, code-smell-based optimization, and compiler-level optimization. First, super-optimization is a technique to find the best

translation of a block of instructions that produce the same result but with lower gas cost. Recent works [14], [15] mainly proposed to convert the gas optimization problem into a MAX-SMT problem and then to use a satisfiability solver to get a reasonable solution. Second, code-smell-based optimization [2]–[12] uses code smells summarized by experts to replace gas-inefficient codes (at both the source code level and the bytecode level) with more efficient codes. Third, compiler-level optimization is an attempt of integrating optimization on bytecodes at the compiler level. In particular, the Solidity compiler [13] has incorporated support for gas optimization at the bytecode level by turning on the *-optimize* option for the *solc* instruction. Overall, the compiler-level optimization is similar to the code-smell-based optimization in that they rely on pre-specified code smells, while super-optimization adopts a very different approach by leveraging satisfiability solvers to discover problem-specific gas-saving solutions.

Applying LLM to software engineering. Since the emergence of LLMs, their tremendous popularity has led to an abundance of published works that utilize these models to execute a variety of tasks. LLMs have been applied across a wide spectrum of software engineering tasks, including Automated Program Repair (APR) [80]–[85], test generation [86]–[90], incident management [91], API recommendation [92], code editing [93], [94], proof generation and repair [95], and fuzzing [96]. These works generally employ LLMs in one of four distinct approaches. Firstly, several studies use direct LLM queries for a carefully formulated domain-specific task, followed by post-processing of the query result [81], [86]. This is the method we adopted in our study. Secondly, some studies conduct comparative analyses to evaluate different language models, datasets, and more [80], [82], [83], [97]. This is a research focus that is fundamentally different from ours. Thirdly, other works integrate an LLM as an intermediate agent into a multi-step pipeline, a task that would otherwise be challenging or time-consuming using conventional methods [85], [87], [90], [92], [96], [98]. Our methodology also falls under this category, as we delegated the labor-intensive part of the gas code-smell-finding pipeline to GPT-4, thereby significantly reducing human effort. Fourthly, some studies attempt to fine-tune one or more LLMs through training on domain-specific datasets [82]–[84], [88], [89], [91], [93]–[95], [99], [100]. This approach requires the utilization of existing datasets or the creation of new ones. However, given the scarcity of existing known code smells and associated codes, it is challenging to directly fine-tune our model on known gas-wasting code smells, which renders this method unsuitable for our task. Overall, our method integrates two of the four prevalent strategies for harnessing LLMs within the field of software engineering.

Prompt tuning. In general, prompts could be divided into two categories: hard prompts and soft prompts. Hard prompts are handcrafted texts that are interpretable by humans, whereas soft prompts consist of continuous-valued vectors that are learned through methods such as backpropagation. Hard prompts are user-friendly and flexible, with higher transferability across different LLMs than soft prompts; however, it is difficult to tune. On the other hand, soft prompts are easier to

optimize but exhibit very low readability. Some of the tuning methods on hard prompts include incorporating a set of trigger words to elicit knowledge from pre-trained language models [59], leveraging reinforcement learning to generate discrete prompts [101], black box tuning [102], and gradient-based discrete optimization [103]. For continuous tuning, some of the methods include prefix tuning by prepending a vector to the model and only train on the prepended vector [104], subset tuning by finding a subset of the pretrained parameters that is critical to downstream tasks [105], side tuning by training a lightweight side network that is fused with the unchanged pre-trained network via summation [106], and adapter modules [107] by adding only a few trainable parameters per task to achieve strong transfer results. Overall, hard prompt tuning and continuous tuning target different types of prompts, and are both difficult due to different reasons, where hard prompts are combinatorically difficult to tune, and continuous prompts require intensive computational resources.

VIII. DATA AVAILABILITY

The data we used in this paper along with our 47-page code smell report is available at https://github.com/jinan789/gas_patterns.

IX. CONCLUSION

In this paper, we have explored the integration of GPT-4 into the pipeline of discovering new gas-wasting code smells for Solidity at the source code level. The input prompt we feed into GPT-4 is carefully calibrated by grounding on insights from existing literature, and we have showed that our discovered set of code smells is novel and gas-efficient. Importantly, we demonstrate further insights via an ablation study on core design components, a discussion of the reasons behind GPT-4's occasional failures, a characterization of the importance of the identified code smells, and an examination of the extensibility of our proposed pipeline. Overall, we have identified 26 new gas-wasting code smells and 13 categories of how GPT-4 failed, where 13 of our identified gas code smells were not presented in prior work. The 26 code smells achieve a saving of deployment cost by 10.534 % and message call cost by 21.528 % in our testing codes, highlighting the significance and effectiveness of our findings.

APPENDIX A

EXECUTION OF A TRANSACTION: WITH AN EMPHASIS ON GAS COSTS

At this section, we present a simplified version of the gas cost mechanism from the Ethereum yellow paper [27], putting an emphasis on gas computation, and omitting other less related components (e.g. substates, etc).

To begin with, an up-front cost v_0 is required to initiate a transaction:

$$v_0 \equiv T_g T_p + T_v \quad (7)$$

where T_g is the maximum amount of gas that could be used to execute the transaction, T_p is the gas price (wei per gas),

and T_v is the number of wei to be transferred. Overall, this up-front cost is similar to a cash deposit, which is in place to ensure that enough money could be paid for the transaction, and the unused amount will be refunded.

Next, when it comes to executing a transaction, there are several components to the total gas price. First, there is the intrinsic gas g_0 , which is to be paid prior to any execution, which establishes the existence of the transaction:

$$g_0 \equiv \sum_{i \in T_i, T_d} \begin{cases} G_{txdatazero}, & \text{if } i = 0 \\ G_{txdatanonzero}, & \text{otherwise} \end{cases} \quad (8)$$

$$+ \begin{cases} G_{txcreate}, & \text{if } T_t = \phi \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

$$+ G_{transaction} \quad (10)$$

$$+ \sum_{j=0}^{\|T_A\|-1} G_{accesslistaddress} \quad (11)$$

$$+ \|T_A[j]_s\| G_{accessliststorage} \quad (12)$$

where the first term (Equation 8) puts different prices on zero and nonzero bytes in the associated data section (4 gas for each zero byte, 16 gas for non-zero ones), the second term (Equation 9) ($T_t = \phi$ for contract creation and "otherwise" for a message call) puts a charge on contract creations (i.e. 32000 gas), the third terms (Equation 10) specifies a universal cost (i.e. 21000 gas) of issuing a transaction, and the fourth term together with the fifth term (Equation 11 and 12) are related to warming up account and storage access.

Second, there is the execution cost. After some preparation work with the intrinsic gas cost deducted, we are ready to enter the execution stage, where the gas cost of the state transition is defined as:

$$g_1 \equiv g - \begin{cases} \Lambda_4(\sigma_0, A^*, S(T), S(T), g, T_p, T_v, \\ T_i, 0, \phi, \top)[1], & \text{if } T_t = \phi \\ \Theta_4(\sigma_0, A^*, S(T), S(T), T_t, T_t, g, \\ T_p, T_v, T_v, T_d, 0, \top)[1], & \text{otherwise} \end{cases} \quad (13)$$

where the variable g is the amount of gas left after deducting the intrinsic gas (i.e. $g \equiv T_g - g_0$), and Λ and Θ are to denote different executions workflows differentiated by contract creation and message call, individually. By indexing the return values of the Λ and Θ functions at one (zero-indexed), we extract the gas remaining after the transaction execution, which then, when subtracted from g , renders the gas consumption. Internally, the gas cost is computed using the general gas cost function (i.e. $C(\sigma, \mu, A, I)$) via the code execution function (i.e. $\Xi(\sigma, g, A, I)$). We omit further details for simplicity, but the high level idea is that the gas cost function returns the gas cost of executing a series of given bytecode instructions under a certain environment. Other variables listed in Equation 13 are miscellaneous parameters related to the transaction.

Third, there is the code deposit cost. This is specifically charged for contract creations due to placing the body codes of the contract into the state:

Fig. 16: Introduction and problem formulation block.

```

1 I want you to help me find new gas-wasting patterns for
  Solidity. Gas is a unique pricing mechanism in
  Solidity, which measures the amount of
  computational resources put into the execution of
  Solidity codes. Gas is paid in money, so saving
  gas is saving money. The goal of gas optimization
  is to find an equivalent code instruction sequence
  that produces the same output while consuming
  less gas. This may misalign from how people
  typically write codes, since the gas pricing
  mechanism is less intuitive and code writers do
  not often keep it in mind. For example, storage
  variables in Solidity are expensive in terms of
  gas, while memory is not, so unnecessary storage
  operations should be avoided when possible, but
  programmers from other languages might not be
  aware of this.

```

$$g_2 \equiv \mathbf{G}_{\text{codedeposit}} \times \|\mathbf{o}\| \quad (14)$$

where $\mathbf{o}$ is the body code of the created contract, and the Equation 14 above specifies that 200 gas (i.e. $\mathbf{G}_{\text{codedeposit}}$) is to be paid for each byte of body codes.

To summarize, the gas cost of executing a transaction is formulated as follows:

$$\text{transaction gas cost} \equiv \begin{cases} \mathcal{D}(C) = g_0 + g_1 + g_2, & \text{if } T_t = \phi \\ \mathcal{M}(C) = g_0 + g_1, & \text{otherwise} \end{cases} \quad (15)$$

where we denote $\mathcal{D}(C)$ by the gas cost of deploying a contract, and $\mathcal{M}(C)$ by the gas cost of sending a message call.

APPENDIX B

THE PROMPTS USED IN THE WORK

Below we list the prompts that were adopted in this work for the main experiment. The introduction and problem formulation block is shown in Figure 16, the few-shot example block is shown in Figure 17, the input codes block is shown in Figure 18, and the self-reflection block is shown in Figure 19.

REFERENCES

- [1] "Breakdown by smart contract languages," <https://defillama.com/languages>, Accessed: 2023-09-12.
- [2] L. Marchesi, M. Marchesi, G. Destefanis, G. Barabino, and D. Tigano, "Design patterns for gas optimization in ethereum," in *2020 IEEE International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*, 2020, pp. 9–15.
- [3] T. Chen, Y. Feng, Z. Li, H. Zhou, X. Luo, X. Li, X. Xiao, J. Chen, and X. Zhang, "Gaschecker: Scalable analysis for discovering gas-inefficient smart contracts," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1433–1448, 2021.
- [4] Q.-T. Nguyen, B. S. Do, T. T. Nguyen, and B.-L. Do, "Gassaver: A tool for solidity smart contract optimization," ser. BSCI '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 125–134. [Online]. Available: <https://doi.org/10.1145/3494106.3528683>
- [5] K. Nelaturu, S. M. Beillahi, F. Long, and A. Veneris, "Smart contracts refinement for gas optimization," in *2021 3rd Conference on Blockchain Research Applications for Innovative Networks and Services (BRAINS)*, 2021, pp. 229–236.
- [6] Y. Chen, Y. Wang, M. Goyal, J. Dong, Y. Feng, and I. Dillig, "Synthesis-powered optimization of smart contracts via data type refactoring," vol. 6, no. OOPSLA2, oct 2022. [Online]. Available: <https://doi.org/10.1145/3563308>
- [7] E. Albert, J. Correas, P. Gordillo, G. Román Díez, and A. Rubio, *GASOL: Gas Analysis and Optimization for Ethereum Smart Contracts*, 04 2020, pp. 118–125.
- [8] A. Di Sorbo, S. Laudanna, A. Vacca, C. A. Visaggio, and G. Canfora, "Profiling gas consumption in solidity smart contracts," *J. Syst. Softw.*, vol. 186, no. C, apr 2022. [Online]. Available: <https://doi.org/10.1016/j.jss.2021.111193>
- [9] T. Chen, X. Li, X. Luo, and X. Zhang, "Under-optimized smart contracts devour your money," in *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2017, pp. 442–446.
- [10] T. Chen, Z. Li, H. Zhou, J. Chen, X. Luo, X. Li, and X. Zhang, "Towards saving money in using smart contracts," in *2018 IEEE/ACM 40th International Conference on Software Engineering: New Ideas and Emerging Technologies Results (ICSE-NIER)*, 2018, pp. 81–84.
- [11] J. Park, D. Lee, and H. In, "Saving deployment costs of smart contracts by eliminating gas-wasteful patterns," *International Journal of Grid and Distributed Computing*, vol. 10, pp. 53–64, 12 2017.
- [12] Q.-P. Kong, Z.-Y. Wang, Y. Huang, X.-P. Chen, X.-C. Zhou, Z.-B. Zheng, and G. Huang, "Characterizing and detecting gas-inefficient patterns in smart contracts," *Patterns in Smart Contracts. J. Comput. Sci. Technol.*, 2022.
- [13] "Solidity-the optimizer," 2023, <https://docs.soliditylang.org/en/v0.8.20/internals/optimizer.html#>, Last accessed on 2023-8-1.
- [14] E. Albert, P. Gordillo, A. Hernández-Cerezo, A. Rubio, and M. A.

Fig. 17: Few-shot example block.

```

1 First, I will give you a list of known gas-wasting
  patterns as an example. The list is numbered, with
  each bullet point specifying one gas-wasting
  pattern and an explanation of why this pattern
  wastes gas. Please be creative and see if my
  provided codes contain any gas-wasting patterns
  beyond the provided ones. The list is as follows:
2
3 1. Repeated computation of the same expression:
  including repetitive reads of the same expensive
  storage variable (e.g. Overuse of expensive
  storage variables). This could be solved by
  introducing an intermediate variable to hold the
  value
4 2. Variable refactoring: by refactoring variable types
  and thus adjusting memory layout, we could save
  gas by saving the amount of storage space needed
5 3. Redundant operations with same effects: redundant
  operations that overwrite the effect of the
  previous operation, making the previous useless
6 4. Pre-computable operations on constants: performing
  comparisons (or other operations) to constants is
  useless and could be pre-computed
7 5. Simplifiable conditional operations: expressions
  that involve logical operations that could be
  simplified to reduce the amount of computational
  workload.
8 6. "Loop fusion." If two loops have the same starting
  and stopping conditions with the same increment
  steps, then we could consider combining them into
  one, if they have no dependencies.
9 7. "Repeated computations in a loop." If there are
  expressions that produce the same outcome in each
  iteration of a loop, we could move it out of the
  loop and execute only once, which saves
  computation.
10 8. "Opaque predicate." If the outcome of a predicate
  could be inferred to be true/false without
  execution, then we could just replace it with the
  corresponding value to save the computation during
  runtime.

```

Fig. 18: Input codes block.

```

1 Next, I will give you a chunk of Solidity codes from
  which you will find gas-inefficient patterns:
2
3 ... (the input codes to be analyzed are inserted here)

```

Fig. 19: Self-reflection block.

```

1 Now I want you to help me find new gas-wasting patterns
  from the above codes and the list of example
  patterns. Please be creative and think outside the
  box beyond the patterns listed above. Please put
  the new gas-wasting patterns in bullet points,
  using Latex syntax. Also please answer the
  following questions for each of the found patterns
  :
2
3 1. How did you find this pattern? Could you explain the
  reasoning process, step by step, behind the
  discovery?
4 2. What should I do, step by step, to fix the found gas
  -wasting patterns? Could you explain the reasoning
  process, step by step, behind the reason why such
  a fix works?
5 3. On a score of 0 to 5, please rate this new pattern
  in terms of how much it is sacrificing readability
  , security, and maintainability in exchange for
  gas optimization (5 is the least impacted. For
  example, "Readability: 5; Security: 4;
  Maintainability: 1" means no effect on security
  while maintainability is largely affected.)
6
7 Here are some additional requirements:
8 1. If you think my prompt to you has any issues (e.g.
  ambiguity or lack of information), please tell me
  how I should improve it the next time.
9 2. Please reflect on your reasoning process and tell me
  how you could do better next time in terms of
  completing the given task. If you found any issues
  , please also make corresponding improvements to
  your output this time and tell me what you did and
  why such improvement is necessary, in a step-by-
  step manner.

```

- Schett, "Super-optimization of smart contracts," vol. 31, no. 4, jul 2022. [Online]. Available: <https://doi.org/10.1145/3506800>
- [15] E. Albert, P. Gordillo, A. Rubio, and M. A. Schett, "Synthesis of super-optimized smart contracts using max-smt," in *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 177–200. [Online]. Available: https://doi.org/10.1007/978-3-030-53288-8_10
 - [16] OpenAI, "Gpt-4 technical report," 2023.
 - [17] Q. Lyu, S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch, "Faithful chain-of-thought reasoning," 2023.
 - [18] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," 2023.
 - [19] L. Kuhn, Y. Gal, and S. Farquhar, "Clam: Selective clarification for ambiguous questions with generative language models," 2023.
 - [20] A. Tamkin, K. Handa, A. Shrestha, and N. Goodman, "Task ambiguity in humans and language models," 2022.
 - [21] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
 - [22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 824–24 837. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ec4f415af0f7b31abca4-Paper-Conference.pdf
 - [23] G. Feng, B. Zhang, Y. Gu, H. Ye, D. He, and L. Wang, "Towards revealing the mystery behind chain of thought: A theoretical perspective," 2023.
 - [24] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei, "Language models are multilingual chain-of-thought reasoners," 2022.
 - [25] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023.
 - [26] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 22 199–22 213. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/8bb0d291acd4acf06ef112099c16f326-Paper-Conference.pdf
 - [27] G. Wood et al., "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, 2023.
 - [28] T. Brandstätter, S. Schulte, J. Cito, and M. Borkowski, "Characterizing efficiency optimizations in solidity smart contracts," in *2020 IEEE International Conference on Blockchain (Blockchain)*, 2020, pp. 281–290.
 - [29] Z. Zhao, J. Li, Z. Su, and Y. Wang, "Gasaver: A static analysis tool for saving gas," *IEEE Transactions on Sustainable Computing*, vol. 8, no. 2, pp. 257–267, 2023.
 - [30] K. Kaur, S. Tomar, and M. Tripathi, "Gas fee reduction by detecting loop fusible patterns in ethereum smart contract," 12 2022, pp. 458–463.
 - [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
 - [32] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," 2023.
 - [33] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evul-instruct," 2023.
 - [34] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023.
 - [35] "Models," <https://platform.openai.com/docs/models/gpt-4-and-gpt-4-turbo>, Accessed: 2024-04-09.
 - [36] "Verified contracts," 2023, <https://etherscan.io/contractsVerified/>, Last accessed on 2023-8-1.
 - [37] A. G. Koru, D. Zhang, K. El Emam, and H. Liu, "An investigation into the functional form of the size-defect relationship for software modules," *IEEE Transactions on Software Engineering*, vol. 35, no. 2, pp. 293–304, 2009.
 - [38] D. Oliveira, W. K. G. Assunção, A. Garcia, A. C. Bibiano, M. Ribeiro, R. Gheyi, and B. Fonseca, "The untold story of code refactoring customizations in practice," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 108–120.
 - [39] C. F. Torres, M. Steichen, and R. State, "The art of the scam: demystifying honeypots in ethereum smart contracts," in *Proceedings of the 28th USENIX Conference on Security Symposium*, ser. SEC'19. USA: USENIX Association, 2019, p. 1591–1607.
 - [40] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," 2022.
 - [41] A. Madaan, N. Tandon, P. Clark, and Y. Yang, "Memory-assisted prompt editing to improve GPT-3 after deployment," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 2833–2861. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.183>
 - [42] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, "Fine-tuning language models from human preferences," 2020.

- [43] N. Stiennon, L. Ouyang, J. Wu, D. M. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. Christiano, "Learning to summarize from human feedback," 2022.
- [44] Y. Tang, Z. Liu, Z. Zhou, and X. Luo, "Chatgpt vs sbst: A comparative assessment of unit test suite generation," 2023.
- [45] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? assessing the quality of code generation by chatgpt," 2024.
- [46] Y. Liu, T. Le-Cong, R. Widyasari, C. Tantithamthavorn, L. Li, X.-B. D. Le, and D. Lo, "Refining chatgpt-generated code: Characterizing and mitigating code quality issues," 2023.
- [47] M. Agrawal, S. Hegselmann, H. Lang, Y. Kim, and D. Sontag, "Large language models are few-shot clinical information extractors," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 1998–2022.
- [48] B. Swanson, K. W. Mathewson, B. Pietrzak, S. Chen, and M. Dinulescu, "Story centaur: Large language model few shot learning as a creative writing tool," in *Conference of the European Chapter of the Association for Computational Linguistics*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:233365238>
- [49] Z. Wang, M. Li, R. Xu, L. Zhou, J. Lei, X. Lin, S. Wang, Z. Yang, C. Zhu, D. Hoiem, S.-F. Chang, M. Bansal, and H. Ji, "Language models with image descriptors are strong few-shot video-language learners," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 8483–8497. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/381ceae4a1feb1abc59c773f7e61839-Paper-Conference.pdf
- [50] C. Larman and V. R. Basili, "Iterative and incremental developments. a brief history," *Computer*, vol. 36, no. 6, pp. 47–56, 2003.
- [51] A. E. Hassan, "Predicting faults using the complexity of code changes," in *2009 IEEE 31st International Conference on Software Engineering*, 2009.
- [52] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering – a systematic literature review," *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, 2009, special Section - Most Cited Articles in 2002 and Regular Research Papers. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584908001390>
- [53] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," 2023.
- [54] B. Kitchenham, L. Madeyski, and D. Budgen, "Segress: Software engineering guidelines for reporting secondary studies," *IEEE Transactions on Software Engineering*, vol. PP, pp. 1–1, 01 2022.
- [55] "The gas patterns report," 2023, https://github.com/jinan789/gas_patterns/blob/main/pattern_report.pdf.
- [56] "The remix ide," <https://remix-ide.readthedocs.io/en/latest/>, Accessed: 2024-04-09.
- [57] "Integrated development environments (ides)," <https://ethereum.org/en/developers/docs/ides/>, Accessed: 2024-04-09.
- [58] "The history of ethereum," <https://ethereum.org/en/history/#shapella>, Accessed: 2024-04-09.
- [59] T. Shin, Y. Razeghi, R. L. L. I. au2, E. Wallace, and S. Singh, "Auto-prompt: Eliciting knowledge from language models with automatically generated prompts," 2020.
- [60] "Vyper," <https://docs.vyperlang.org/en/stable/>, Accessed: 2024-04-25.
- [61] "Eip-7609: Decrease base cost of tload/tstore," <https://eips.ethereum.org/EIPS/eip-7609>, Accessed: 2024-04-25.
- [62] "Eip-1153: Transient storage opcodes," <https://eips.ethereum.org/EIPS/eip-1153>, Accessed: 2024-04-25.
- [63] "Anthropic," <https://www.anthropic.com/>, Accessed: 2024-04-21.
- [64] "Introducing the next generation of claude," <https://www.anthropic.com/news/claude-3-family>, Accessed: 2024-04-21.
- [65] "Unix_timestamp," https://mariadb.com/kb/en/unix_timestamp/, Accessed: 2023-08-18.
- [66] "Bson types," <https://www.mongodb.com/docs/v4.4/reference/bson-types/>, Accessed: 2023-08-18.
- [67] "Appendix - dynamic gas costs," <https://github.com/wolflo/evm-opcodes/blob/main/gas.md>, Accessed: 2023-08-18.
- [68] "Opcodes for the evm," <https://ethereum.org/en/developers/docs/evm/opcodes/>, Accessed: 2023-08-18.
- [69] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.
- [70] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, Y. Wang, X. Lin, T. Chen, and Z. Zheng, "When chatgpt meets smart contract vulnerability detection: How far are we?" 2023.
- [71] A. M. Ebrahimi, G. A. Oliva, and A. E. Hassan, "Self-admitted technical debt in ethereum smart contracts: A large-scale exploratory study," *IEEE Transactions on Software Engineering*, vol. 49, no. 9, pp. 4304–4323, 2023.
- [72] A. Jbara and D. G. Fietelson, "How programmers read regular code: a controlled experiment using eye tracking," *Empirical Softw. Engg.*, vol. 22, no. 3, p. 1440–1477, jun 2017. [Online]. Available: <https://doi.org/10.1007/s10664-016-9477-x>
- [73] —, "On the effect of code regularity on comprehension," in *Proceedings of the 22nd International Conference on Program Comprehension*, ser. ICPC 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 189–200. [Online]. Available: <https://doi.org/10.1145/2597008.2597140>
- [74] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024.
- [75] Y. Zhou, X. Geng, T. Shen, C. Tao, G. Long, J.-G. Lou, and J. Shen, "Thread of thought unraveling chaotic contexts," 2023.
- [76] Y. Yao, Z. Li, and H. Zhao, "Beyond chain-of-thought, effective graph-of-thought reasoning in language models," 2024.
- [77] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," 2023.
- [78] J. Long, "Large language model guided tree-of-thought," 2023.
- [79] W. Yu, H. Zhang, X. Pan, K. Ma, H. Wang, and D. Yu, "Chain-of-note: Enhancing robustness in retrieval-augmented language models," 2023.
- [80] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 1482–1494. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00129>
- [81] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 1469–1481. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00128>
- [82] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 1430–1442. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICSE48619.2023.00125>
- [83] K. Huang, X. Meng, J. Zhang, Y. Liu, W. Wang, S. Li, and Y. Zhang, "An empirical study on fine-tuning large language models of code for automated program repair," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
- [84] C. S. Xia, Y. Ding, and L. Zhang, "Revisiting the plastic surgery hypothesis via large language models," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
- [85] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," in *Proceedings of the 2023 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23, 2023.
- [86] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring llm-based general bug reproduction," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 2312–2323. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00194>
- [87] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 919–931. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00085>
- [88] N. Rao, K. Jain, U. Alon, C. L. Goues, and V. J. Hellendoorn, "Cat-Im: Training language models on aligned code and tests," in *Proceedings of*

- the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
- [89] M. Sun, Y. Yang, Y. Wang, M. Wen, H. Jia, and Y. Zhou, "Smt solver validation empowered by large pre-trained language models," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
 - [90] T.-O. Li, W. Zong, Y. Wang, H. Tian, Y. Wang, S.-C. Cheung, and J. Kramer, "Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
 - [91] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23, 2023.
 - [92] Q. Huang, Z. Wan, Z. Xing, C. Wang, J. Chen, X. Xu, and Q. Lu, "Let's chat to find the apis: Connecting human, llm and knowledge graph through ai chain," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
 - [93] P. Gupta, A. Khare, Y. Bajpai, S. Chakraborty, S. Gulwani, A. Kanade, A. Radhakrishna, G. Soares, and A. Tiwari, "Grace: Generation using associated code edits," in *Proceedings of the 2023 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23, 2023.
 - [94] J. Zhang, P. Nie, J. J. Li, and M. Gligoric, "Multilingual code co-evolution using large language models," in *Proceedings of the 2023 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23, 2023.
 - [95] E. First, M. N. Rabe, T. Ringer, and Y. Brun, "Baldur: Whole-proof generation and repair with large language models," in *Proceedings of the 2023 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23, 2023.
 - [96] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 423–435. [Online]. Available: <https://doi.org/10.1145/3597926.3598067>
 - [97] S. Gao, X.-C. Wen, C. Gao, W. Wang, H. Zhang, and M. R. Lyu, "What makes good in-context demonstrations for code intelligence tasks with llms?" in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
 - [98] Z. Tang, J. Ge, S. Liu, T. Zhu, T. Xu, L. Huang, and B. Luo, "Domain adaptive code completion via language models and decoupled domain databases," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
 - [99] X. Zhou, K. Kim, B. Xu, J. Liu, D. Han, and D. Lo, "The devil is in the tails: How long-tailed code distributions impact large language models," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, 2023.
 - [100] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "On the usage of continual learning for out-of-distribution generalization in pre-trained language models of code," in *Proceedings of the 2023 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23, 2023.
 - [101] M. Deng, J. Wang, C.-P. Hsieh, Y. Wang, H. Guo, T. Shu, M. Song, E. Xing, and Z. Hu, "RLPrompt: Optimizing discrete text prompts with reinforcement learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 3369–3391. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.222>
 - [102] S. Diao, Z. Huang, R. Xu, X. Li, Y. Lin, X. Zhou, and T. Zhang, "Black-box prompt learning for pre-trained language models," 2023.
 - [103] Y. Wen, N. Jain, J. Kirchenbauer, M. Goldblum, J. Geiping, and T. Goldstein, "Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery," 2023.
 - [104] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," 2021.
 - [105] M. Zhao, T. Lin, F. Mi, M. Jaggi, and H. Schütze, "Masking as an efficient alternative to finetuning for pretrained language models," 2020.
 - [106] J. O. Zhang, A. Sax, A. Zamir, L. Guibas, and J. Malik, "Side-tuning: A baseline for network adaptation via additive side networks," 2020.
 - [107] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," 2019.